Universidade Federal de Viçosa

Campus **Rio Paranaíba**

João Pedro Silva Coelho - 7022

IMPLEMENTAÇÃO E ANÁLISE DOS ALGORITMOS INSERTION, SELECTION, BUBBLE, SHELL E MERGE SORT

Rio Paranaíba - MG

2026

Universidade Federal de Viçosa

Campus **Rio Paranaíba**

João Pedro Silva Coelho - 7022

IMPLEMENTAÇÃO E ANÁLISE DOS ALGORITMOS INSERTION, SELECTION, BUBBLE, SHELL E MERGE SORT

Trabalho apresentado para obtenção de créditos na disciplina de Projeto de Algoritmos da Universidade Federal de Viçosa - Campus de Rio Paranaíba.

Rio Paranaíba - MG

2026

1 RESUMO

Este trabalho tem como objetivo implementar e analisar cinco algoritmos clássicos de ordenação por comparação estudados na disciplina de Projeto de Algoritmos: Insertion Sort, Selection Sort, Bubble Sort, Shell Sort e Merge Sort. Foi desenvolvida uma aplicação em C++, com criação automática das pastas e arquivos pelo próprio programa, capaz de gerar instâncias de teste de diferentes tamanhos (10, 100, 1.000, 10.000, 100.000 e 1.000.000 de elementos) e diferentes características (crescente, decrescente e randômica), executar cada algoritmo sobre elas e registrar o tempo de execução observado.

Os resultados foram organizados em arquivos de entrada, saída e tempo para cada algoritmo, consolidados em tabelas e gráficos individuais e comparativos que permitem analisar o comportamento de cada algoritmo isoladamente e compará-los entre si. As medições confirmam o comportamento teórico esperado para cada algoritmo: desempenho linear no melhor caso do Insertion e do Bubble Sort (entrada já ordenada), desempenho quadrático em qualquer caso para o Selection Sort, e uma clara superioridade dos algoritmos de menor complexidade: para 1.000.000 de elementos randômicos, o Merge Sort (0,078 s) e o Shell Sort (0,106 s) foram cerca de mil a doze mil vezes mais rápidos que os três algoritmos $O(n^2)$ (80,6 s a 938,7 s). O Merge Sort, cujo custo é $\Theta(n \log n)$ independentemente da ordem inicial dos dados, foi o mais rápido para entradas randômicas, ao passo que o Shell Sort foi o mais rápido para entradas já ordenadas ou decrescentes, evidenciando a forte influência da estratégia de cada algoritmo — e, no caso do Insertion, Selection e Bubble Sort, também do grau de ordenação inicial dos dados — sobre o desempenho observado.

Sumário

1 RESUMO **1**

2 INTRODUÇÃO **4**

3 ALGORITMOS **5**

3.1 Insertion Sort 5

3.2 Teste de Mesa Manual 6

3.3 Selection Sort 7

3.4 Teste de Mesa Manual 8

3.5 Bubble Sort 10

3.6 Teste de Mesa Manual 11

3.7 Shell Sort 13

3.8 Teste de Mesa Manual 14

3.9 Merge Sort 17

3.10 Teste de Mesa Manual 19

4 ANÁLISE DE COMPLEXIDADE **21**

4.1 Insertion Sort 21

4.2 Selection Sort 23

4.3 Bubble Sort 25

4.4 Shell Sort 27

4.5 Merge Sort 29

5 TABELA E GRÁFICO **32**

5.1 Insertion Sort 32

5.2 Selection Sort 34

5.3 Bubble Sort 36

5.4 Shell Sort 38

5.5 Merge Sort 40

6 TABELA DE COMPARAÇÃO **42**

7 GRÁFICO GERAL **44**

8 CONCLUSÃO **46**

9 REFERÊNCIAS BIBLIOGRÁFICAS **48**

10 Função para calcular o tempo **48**

2 INTRODUÇÃO

Em várias situações do cotidiano é necessário manter conjuntos de dados organizados: da lista telefônica ao histórico de transações bancárias, a ordenação é uma operação central em ciência da computação. Diante disso, diversos algoritmos de ordenação foram desenvolvidos ao longo dos anos, cada um com características próprias de desempenho, simplicidade de implementação e comportamento em diferentes cenários de entrada.

Este trabalho é dividido em cinco capítulos, um para cada algoritmo de ordenação estudado: o Capítulo 1 trata do Insertion Sort, o Capítulo 2 do Selection Sort, o Capítulo 3 do Bubble Sort, o Capítulo 4 do Shell Sort e o Capítulo 5 do Merge Sort. Os quatro primeiros são algoritmos de ordenação por comparação amplamente utilizados como introdução ao estudo de algoritmos e possuem complexidade $O(n^2)$ no pior caso (no caso do Shell Sort, com a sequência de intervalos original utilizada neste trabalho, e apenas para entradas construídas especificamente para isso) — ainda que, dependendo dos dados de entrada, alguns deles possam ser bem mais rápidos do que essa cota sugere. O Merge Sort, por sua vez, baseia-se no paradigma de dividir para conquistar e tem complexidade $\Theta(n \log n)$ em todos os casos, ao custo de utilizar memória adicional proporcional ao tamanho da entrada.

Ao longo de cada capítulo são apresentados o funcionamento do respectivo algoritmo, um teste de mesa manual, a análise de complexidade nas diferentes situações de entrada, e os resultados experimentais obtidos a partir de uma implementação em C++, testada com instâncias de tamanhos crescentes (de 10 a 1.000.000 de elementos) e três padrões de entrada: crescente, decrescente e randômica. Ao final do trabalho, os cinco algoritmos são comparados entre si por meio de uma tabela e um gráfico geral, seguidos de uma conclusão consolidada sobre os resultados observados.

3 ALGORITMOS

3.1 Insertion Sort

O Insertion Sort é um algoritmo de ordenação por comparação que constrói a lista ordenada de forma incremental, um elemento por vez. Sua lógica é semelhante à forma como uma pessoa organiza cartas de baralho na mão: a cada nova carta recebida, ela é inserida na posição correta em relação às cartas que já estão ordenadas.

O algoritmo percorre o vetor a partir do segundo elemento (índice 1). A cada iteração, o elemento da posição atual, chamado de *chave*, é comparado com os elementos anteriores, que já se encontram ordenados entre si. Enquanto houver, à esquerda da chave, elementos maiores que ela, esses elementos são deslocados uma posição à frente, abrindo espaço para a chave ser inserida na posição correta. Esse processo se repete até que todo o vetor tenha sido percorrido, restando ao final um vetor completamente ordenado em ordem crescente.

A seguir apresenta-se a implementação utilizada neste trabalho, em C++, correspondente à função `InsertionSort`:

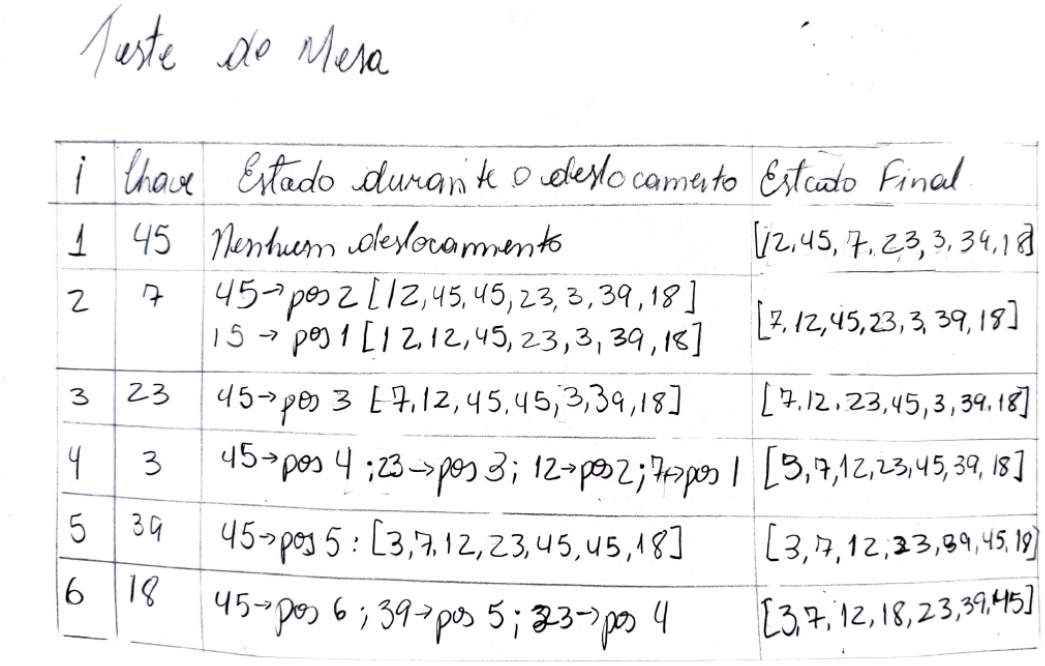
```
void InsertionSort(int *vetor, int tamanho) {
    int chave, j;
    for (int i = 1; i < tamanho; i++) {
        chave = vetor[i];
        j = i - 1;
        while (j >= 0 && vetor[j] > chave) {
            vetor[j + 1] = vetor[j];
            j--;
        }
        vetor[j + 1] = chave;
    }
}
```

Uma característica importante do Insertion Sort é que seu desempenho depende diretamente do grau de ordenação da entrada: quanto mais próxima a entrada já estiver da ordem

final, menos deslocamentos serão necessários durante a execução. Essa característica é explorada em detalhes na análise de complexidade e nos resultados experimentais apresentados nas próximas seções deste capítulo.

3.2 Teste de Mesa Manual

O teste de mesa manual do Insertion Sort, apresentado na figura abaixo, foi realizado à mão com um exemplo de entrada diferente dos exemplos trabalhados em sala de aula, conforme exigido pelo enunciado do trabalho.



Teste de Mesa

i	Chave	Estado durante o deslocamento	Estado Final
1	45	Nenhum deslocamento	[12, 45, 7, 23, 3, 39, 18]
2	7	45→pos 2 [12, 45, 45, 23, 3, 39, 18] 12→pos 1 [12, 12, 45, 23, 3, 39, 18]	[7, 12, 45, 23, 3, 39, 18]
3	23	45→pos 3 [7, 12, 45, 45, 3, 39, 18]	[7, 12, 23, 45, 3, 39, 18]
4	3	45→pos 4; 23→pos 3; 12→pos 2; 7→pos 1	[3, 7, 12, 23, 45, 39, 18]
5	39	45→pos 5: [3, 7, 12, 23, 45, 45, 18]	[3, 7, 12, 23, 39, 45, 18]
6	18	45→pos 6; 39→pos 5; 23→pos 4	[3, 7, 12, 18, 23, 39, 45]

Figura 1: Teste de mesa manual do algoritmo Insertion Sort

3.3 Selection Sort

O Selection Sort é um algoritmo de ordenação por comparação baseado na ideia de, repetidamente, selecionar o menor elemento ainda não ordenado e colocá-lo em sua posição final. É um dos algoritmos mais intuitivos: em vez de inserir elementos em uma sequência já ordenada (como faz o Insertion Sort), o Selection Sort constrói a parte ordenada do vetor da esquerda para a direita, sempre buscando, entre os elementos restantes, aquele que deve ocupar a próxima posição.

O algoritmo percorre o vetor com um índice i, de 0 até tamanho-2. Para cada posição i, é feita uma busca linear pelo menor elemento entre as posições i e tamanho-1. Encontrado o menor elemento, ele é trocado de lugar com o elemento da posição i (a troca só é realizada se o menor encontrado não estiver já na posição i). Repetindo esse processo para todas as posições, o vetor fica completamente ordenado em ordem crescente ao final da execução.

A seguir apresenta-se a implementação utilizada neste trabalho, em C++, correspondente à função `SelectionSort`:

```
void SelectionSort(int *vetor, int tamanho) {
    int menor, aux;
    for (int i = 0; i < tamanho - 1; i++) {
        menor = i;
        for (int j = i + 1; j < tamanho; j++) {
            if (vetor[j] < vetor[menor]) {
                menor = j;
            }
        }
        if (menor != i) {
            aux = vetor[i];
            vetor[i] = vetor[menor];
            vetor[menor] = aux;
        }
    }
```

```
}
```

Uma característica marcante do Selection Sort, explorada em detalhes na análise de complexidade deste capítulo, é que o número de *comparações* realizadas não depende de a entrada já estar ordenada, invertida ou embaralhada: a busca pelo menor elemento sempre percorre integralmente os elementos restantes. Isso o diferencia do Insertion Sort e do Bubble Sort, que se beneficiam de entradas parcialmente ordenadas. A única grandeza que varia com o tipo de entrada é o número de *trocas*, que é, no máximo, tamanho-1 em qualquer caso — uma quantidade desprezível diante do número de comparações.

3.4 Teste de Mesa Manual

O teste de mesa manual do Selection Sort, apresentado na figura abaixo, foi realizado à mão com um exemplo de entrada diferente dos exemplos trabalhados em sala de aula, conforme exigido pelo enunciado do trabalho.

Selection sort

Teste de mesa : [29, 10, 14, 37, 3, 22, 8]

i	Busca do menor	Índice menor	Troca realizada	Vetor final
0	V = [1 a 6] com V[0] = 29	4 (valor 3)	V[0] ↔ V[4]	[3, 10, 14, 37, 29, 22, 8]
1	V = [2 a 6] com V = [1] = 10	6 (valor 8)	V[1] ↔ V[6]	[3, 8, 14, 37 29, 22, 10]
2	V = [3 a 6] com V = [2] = 14	6 (valor 10)	V[2] ↔ V[6]	[3, 8, 10, 37, 29, 22, 14]
3	V = [4 a 5] com V = [3] = 37	6 (valor 10)	V[3] ↔ V[6]	[3, 8, 10, 14, 29, 22, 37]
4	V = [5 a 6] com V = [4] = 29	5 (valor 22)	V[4] ↔ V[5]	[3, 8, 10, 14, 22, 29 37]
5	V = [6] com V = [5] = 29	5 (valor 29	não troca	[3, 8, 10, 14, 22, 29, 37]

Figura 2: Teste de mesa manual do algoritmo Selection Sort

3.5 Bubble Sort

O Bubble Sort (ou "ordenação por bolha") é um algoritmo de ordenação por comparação que percorre repetidamente o vetor, comparando pares de elementos *adjacentes* e trocando-os de posição sempre que estiverem fora de ordem. A cada passada completa pelo vetor, o maior elemento ainda não posicionado "borbulha"até a sua posição final, de forma análoga a uma bolha de ar subindo até a superfície — daí o nome do algoritmo.

O algoritmo realiza, no máximo, tamanho-1 passadas. Em cada passada i, percorre-se o vetor comparando os pares (vetor[j], vetor[j+1]), trocando-os quando vetor[j] ¿ vetor[j+1]. A versão implementada neste trabalho inclui uma otimização amplamente utilizada na prática: uma variável de controle (*trocou*) que registra se alguma troca ocorreu durante a passada. Caso nenhuma troca tenha ocorrido, o vetor já está ordenado e o algoritmo é interrompido imediatamente, sem realizar as passadas restantes — isso é o que torna o comportamento do algoritmo linear quando a entrada já está ordenada.

A seguir apresenta-se a implementação utilizada neste trabalho, em C++, correspondente à função `BubbleSort`:

```
void BubbleSort(int *vetor, int tamanho) {
    int aux;
    bool trocou;
    for (int i = 0; i < tamanho - 1; i++) {
        trocou = false;
        for (int j = 0; j < tamanho - 1 - i; j++) {
            if (vetor[j] > vetor[j + 1]) {
                aux = vetor[j];
                vetor[j] = vetor[j + 1];
                vetor[j + 1] = aux;
                trocou = true;
            }
        }
        if (!trocou) break;
```

```
    }
}
```

Assim como o Insertion Sort, o Bubble Sort se beneficia de entradas já ordenadas ou quase ordenadas, apresentando desempenho linear nesse cenário graças à parada antecipada. Por outro lado, em entradas decrescentes ou aleatórias, o algoritmo realiza um número de comparações da mesma ordem de grandeza do Insertion Sort, porém com um custo maior por troca (três atribuições contra uma no deslocamento do Insertion Sort), o que tende a torná-lo mais lento na prática, efeito que os resultados experimentais deste capítulo mostram com clareza no caso de entradas aleatórias, como discutido na análise de complexidade.

3.6 Teste de Mesa Manual

O teste de mesa manual do Bubble Sort, apresentado na figura abaixo, foi realizado à mão com um exemplo de entrada diferente dos exemplos trabalhados em sala de aula, conforme exigido pelo enunciado do trabalho.

Bubble Sort

Teste de mesa V=[8, 34, 21, 5, 47, 12, 3]

i	Trocas realizadas	Vetor final	Troca?
0	(8,34) sim troca (34,21) troca (34,5) troca (34,47) não troca (47,12) troca (47,3) troca	[8,21,5,34,12,3,47]	Sim
1	(8,21) não (21,5) troca (21,34) não (34,12) troca (34,3) troca (34,47) não	[8,5,21,12,3,34,47]	sim
2	(8,5) troca (8,21) não (21,12) troca (21,3) troca (21,34) não (34,47) não	[5,8,12,3,21,34,47]	Sim
3	(5,8) não (8,12) não (12,3) troca (12,21) não (21,34) não (34,47) não	[5,8,3,12,21,34,47]	sim
4	(5,8) não (8,3) troca (8,12) não (12,21) não (21,34) não (34,47) não	[5,3,8,12,21,34,47]	sim
5	(5,3) troca (5,8) não (8,12) não (12,21) não (21,34) não (34,47) não	[3,5,8,12,21,34,47]	sim

Figura 3: Teste de mesa manual do algoritmo Bubble Sort

3.7 Shell Sort

O Shell Sort é um algoritmo de ordenação por comparação que pode ser entendido como uma generalização do Insertion Sort. Em vez de comparar e deslocar apenas elementos adjacentes desde o início, o Shell Sort primeiro ordena, de forma aproximada, subconjuntos de elementos espaçados por um intervalo (chamado de *gap*), e vai reduzindo progressivamente esse intervalo até chegar a 1 — quando o algoritmo se comporta exatamente como um Insertion Sort tradicional, porém sobre um vetor que já está quase ordenado, o que reduz drasticamente o número de deslocamentos necessários nessa última etapa.

A implementação utilizada neste trabalho adota a sequência de intervalos originalmente proposta por Donald Shell: o primeiro intervalo é tamanho/2, e a cada passada o intervalo é dividido por 2 (divisão inteira), até chegar a 1. Para cada intervalo (gap), aplica-se a lógica do Insertion Sort a cada um dos subvetores formados por elementos espaçados de *gap* posições entre si.

A seguir apresenta-se a implementação utilizada neste trabalho, em C++, correspondente à função `ShellSort`:

```
void ShellSort(int *vetor, int tamanho) {
    int chave, j;
    for (int gap = tamanho / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < tamanho; i++) {
            chave = vetor[i];
            j = i - gap;
            while (j >= 0 && vetor[j] > chave) {
                vetor[j + gap] = vetor[j];
                j -= gap;
            }
            vetor[j + gap] = chave;
        }
    }
}
```

A escolha da sequência de intervalos é um dos aspectos mais estudados do Shell Sort, pois afeta diretamente sua complexidade de pior caso — esse ponto é discutido em detalhes, com base em pesquisa bibliográfica, na seção de análise de complexidade deste capítulo, conforme solicitado no enunciado do trabalho. De forma geral, mesmo utilizando a sequência mais simples (a original de Shell), o algoritmo apresenta desempenho muito superior ao Insertion, Selection e Bubble Sort na prática, para todos os tipos de entrada testados.

3.8 Teste de Mesa Manual

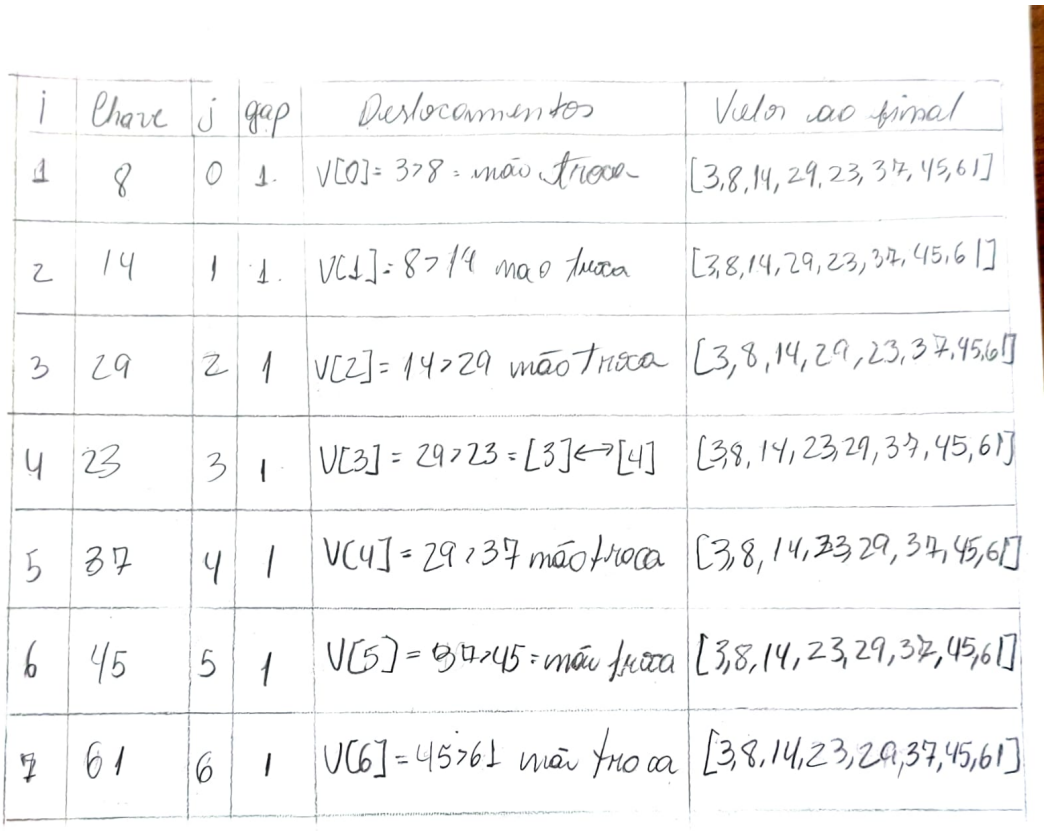
O teste de mesa manual do Shell Sort, apresentado na figura abaixo, foi realizado à mão com um exemplo de entrada diferente dos exemplos trabalhados em sala de aula, conforme exigido pelo enunciado do trabalho. Devido ao maior número de passadas do Shell Sort (uma para cada intervalo utilizado na execução), o teste de mesa não coube em uma única folha e está dividido em duas partes: a primeira contempla as passadas com os intervalos gap = 4 e gap = 2, e a segunda contempla a passada final, com gap = 1.

Shell sort

Teste de mesa: [45, 8, 23, 61, 3, 37, 14, 29]

i	chave	j	gap	Deslocamento	Vetor após final
4	3	0	4	V[0]=45>3 = [0]↔[4]	[3,8,23,61,45,37,14,29]
5	37	1	4	V[1]=8>37 = não troca	[3,8,23,61,45,37,14,29]
6	14	2	4	V[2]=23>14 = [2]↔[6]	[3,8,14,61,45,37,23,29]
7	29	3	4	V[3]=61>29 = [3]↔[7]	[3,8,14,29,45,37,23,61]
2	14	0	2	V[0]=3>14 não troca	[3,8,14,29,45,37,23,61]
3	29	1	2	V[1]=8>29 não troca	[3,8,14,29,45,37,23,61]
4	45	2	2	V[2] 14>45 não troca	[3,8,14,29,45,37,23,61]
5	37	3	2	V[3]=29>37 = não troca	[3,8,14,29,45,37,23,61]
6	23	4	2	V[4]=45>23 [4]↔[6]	[3,8,14,29,23,37,45,61]
7	6	5	2	V[5]=37>61 não troca	[3,8,14,29,23,37,45,61]

Figura 4: Teste de mesa manual do algoritmo Shell Sort – parte 1 de 2 (passadas com gap = 4 e gap = 2)



i	Chave	j	gap	Deslocamentos	Vetor ao final
1	8	0	1	V[0] = 3 > 8 = não troca	[3,8,14,29,23,37,45,61]
2	14	1	1	V[1] = 8 > 14 não troca	[3,8,14,29,23,37,45,61]
3	29	2	1	V[2] = 14 > 29 não troca	[3,8,14,29,23,37,45,61]
4	23	3	1	V[3] = 29 > 23 = [3] ↔ [4]	[3,8,14,23,29,37,45,61]
5	37	4	1	V[4] = 29 > 37 não troca	[3,8,14,23,29,37,45,61]
6	45	5	1	V[5] = 37 > 45 = não troca	[3,8,14,23,29,37,45,61]
7	61	6	1	V[6] = 45 > 61 não troca	[3,8,14,23,29,37,45,61]

Figura 5: Teste de mesa manual do algoritmo Shell Sort – parte 2 de 2 (passada com gap = 1)

3.9 Merge Sort

O Merge Sort (ordenação por intercalação) é um algoritmo de ordenação por comparação baseado no paradigma de *dividir para conquistar*: em vez de ordenar o vetor inteiro de uma só vez, ele o divide em partes menores, ordena cada parte separadamente e, por fim, combina os resultados. A ideia central é que *intercalar* (*merge*) dois vetores que já estão ordenados é uma operação simples e eficiente, pois basta comparar os elementos que estão no início de cada um e ir retirando sempre o menor deles. Segundo Knuth (1998), o método foi proposto por John von Neumann em 1945.

O algoritmo é executado, de forma recursiva, em três etapas:

1. **Dividir:** o vetor é dividido ao meio, em duas metades de tamanhos (aproximadamente) iguais;
2. **Conquistar:** cada metade é ordenada recursivamente, aplicando o próprio Merge Sort. A recursão termina quando o trecho possui um único elemento (ou nenhum), que já está ordenado por definição;
3. **Combinar:** as duas metades, agora ordenadas, são intercaladas em um vetor auxiliar: compara-se o primeiro elemento ainda não utilizado de cada metade, copia-se o menor deles para o vetor auxiliar e avança-se naquela metade; quando uma das metades se esgota, copia-se o restante da outra; por fim, o trecho intercalado é copiado de volta para o vetor original.

A seguir apresenta-se a implementação utilizada neste trabalho, em C++, composta por três funções: `Intercalar`, que realiza a etapa de combinação; `MergeSortRecursivo`, que realiza a divisão e as chamadas recursivas; e `MergeSort`, ponto de entrada com a mesma assinatura dos demais algoritmos do trabalho (vetor e tamanho), que aloca o vetor auxiliar uma única vez:

```
void Intercalar(int *vetor, int *aux,
                int inicio, int meio, int fim) {
    int i = inicio, j = meio + 1, k = inicio;
```

```
    while (i <= meio && j <= fim) {
        if (vetor[i] <= vetor[j])
            aux[k++] = vetor[i++];
        else
            aux[k++] = vetor[j++];
    }
    while (i <= meio) aux[k++] = vetor[i++];
    while (j <= fim)  aux[k++] = vetor[j++];
    for (k = inicio; k <= fim; k++)
        vetor[k] = aux[k];
}

void MergeSortRecursivo(int *vetor, int *aux,
                        int inicio, int fim) {
    if (inicio >= fim) return;
    int meio = inicio + (fim - inicio) / 2;
    MergeSortRecursivo(vetor, aux, inicio, meio);
    MergeSortRecursivo(vetor, aux, meio + 1, fim);
    Intercalar(vetor, aux, inicio, meio, fim);
}

void MergeSort(int *vetor, int tamanho) {
    if (tamanho < 2) return;
    int *aux = new int[tamanho];
    MergeSortRecursivo(vetor, aux, 0, tamanho - 1);
    delete[] aux;
}
```

Algumas características importantes do Merge Sort, que o distinguem dos demais algoritmos deste trabalho:

- **Desempenho previsível:** como a divisão e a intercalação são sempre executadas por completo, o custo é da ordem de $n \log n$ qualquer que seja a ordem inicial dos dados, ao contrário do Insertion e do Bubble Sort, que dependem do grau de ordenação da entrada;

- **Estabilidade:** o uso de `<=` na comparação faz com que, em caso de empate, o elemento da metade esquerda seja copiado primeiro, preservando a ordem relativa de elementos iguais;

- **Memória adicional:** é necessário um vetor auxiliar do mesmo tamanho do vetor original (cerca de 4 MB para 1.000.000 de inteiros), enquanto os outros quatro algoritmos ordenam no próprio vetor;

- **Profundidade da recursão:** como o vetor é dividido ao meio a cada chamada, a profundidade é de cerca de $\log_2 n$ níveis (20 níveis para 1.000.000 de elementos), o que não representa risco de estouro de pilha.

Essas características são exploradas em detalhes na análise de complexidade e nos resultados experimentais apresentados nas próximas seções deste capítulo.

3.10 Teste de Mesa Manual

O teste de mesa manual do Merge Sort, apresentado na figura abaixo, foi realizado à mão com um exemplo de entrada diferente dos exemplos trabalhados em sala de aula, conforme exigido pelo enunciado do trabalho, e mostra o estado do vetor a cada divisão e a cada intercalação realizada.

Teste de mesa - Merge sort [26,9,41,3,18,33,12]

ORDEM	Início/Fim	MEIO	METADE DIREITA	METADE ESQUERDA
1	[0...6]	3	[26,9,41,3]	[18,33,12]
2	[0..3]	1	[26,9]	[41,3]
3	[0...1]	0	[26]	[9]
4	[2...3]	2	[41]	[3]
5	[4...6]	5	[18,33]	[12]
6	[4...5]	4	[18]	[33]

Combinação

ORDEM	TRECHO	ESQ + DIR.	REDUZIDO	COMPARAÇÕES	VETOR FINAL
1	[0..1]	[26]+[9]	[9,26]	1	[9,26,41,3,18,33,12]
2	[2..3]	[41]+[3]	[3,41]	1	[9,26,3,41,18,33,12]
3	[0...3]	[9,26]+[3,41]	[3,9,26,41]	3	[3,9,26,41,18,33,12]
4	[4...5]	[18]+[33]	[18,33]	1	[3,9,26,41,18,33,12]
5	[4..6]	[18,33]+[9]	[12,18,33]	1	[3,9,26,41,12,18,33]
6	[0..6]	[3,9,26,41]+[12,18,33]	[3,9,12,18,26,33,41]	6	[3,9,12,18,26,33,41]

Figura 6: Teste de mesa manual do algoritmo Merge Sort

4 ANÁLISE DE COMPLEXIDADE

4.1 Insertion Sort

A análise de complexidade do Insertion Sort, considerando as diferentes situações de entrada (crescente, decrescente e randômica), foi realizada manuscrita, com a devida justificativa, contando o número de execuções de cada linha do algoritmo (associando uma constante de custo c_i a cada linha) e somando essas contagens em função de n, conforme exigido pelo enunciado do trabalho.

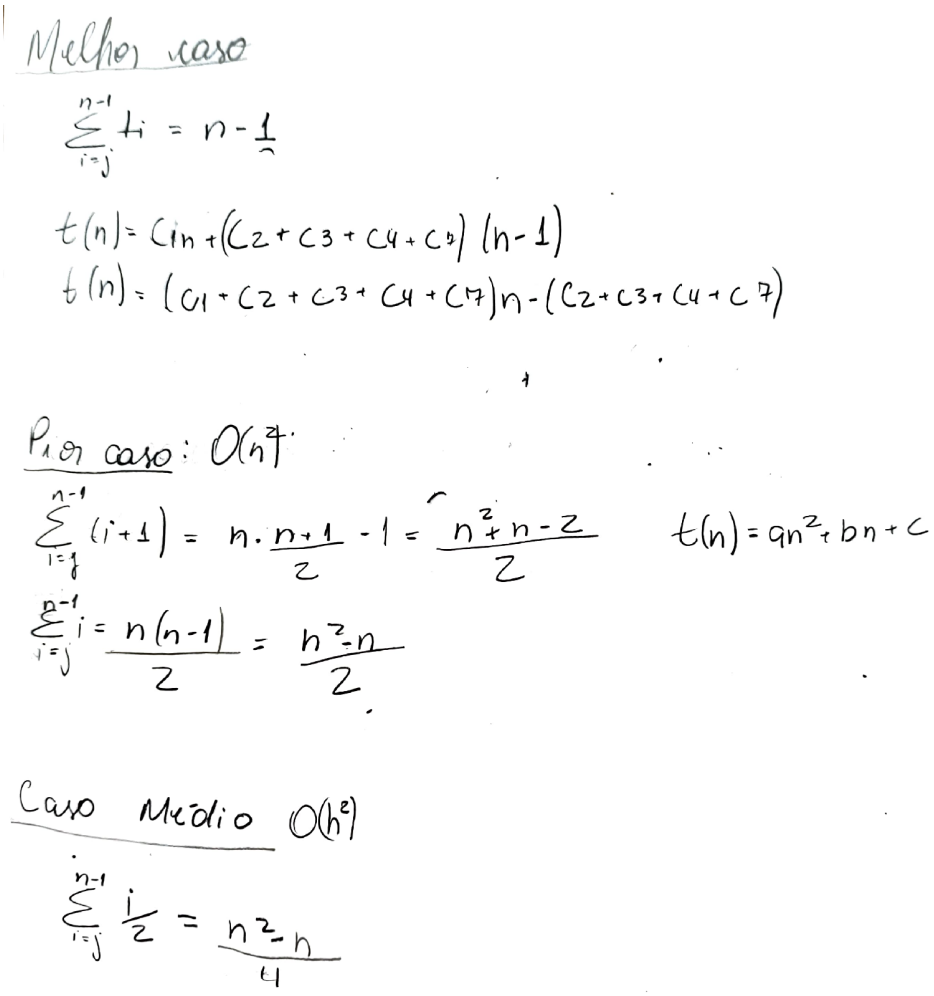


Figura 7: Análise de complexidade manuscrita do algoritmo Insertion Sort

Em resumo, a análise manuscrita chega às seguintes conclusões para cada situação de entrada:

Melhor caso (entrada crescente): o laço interno nunca desloca elementos, pois cada elemento já está na posição correta. A contagem resulta em $T(n) = (c_1 + c_2 + c_3 + c_4 + c_7)\, n - (c_2 + c_3 + c_4 + c_7)$, uma função linear em n, ou seja, $T(n) = O(n)$.

Pior caso (entrada decrescente): cada elemento precisa ser deslocado até o início da parte já ordenada, resultando em $\sum_{i=1}^{n-1}(i+1) = \dfrac{n^2 + n - 2}{2}$ deslocamentos. A contagem tem a forma $T(n) = an^2 + bn + c$, portanto $T(n) = \Theta(n^2)$.

Caso médio (entrada randômica): em média, cada elemento precisa ser deslocado apenas até a metade da parte já ordenada ($\sum_{i=1}^{n-1} \frac{i}{2} = \frac{n^2-n}{4}$ deslocamentos esperados), de modo que o caso médio também é $T(n) = \Theta(n^2)$, com aproximadamente metade do trabalho do pior caso.

Os resultados experimentais apresentados na próxima seção deste capítulo são compatíveis com essa análise teórica: desempenho linear no melhor caso (entrada crescente) e desempenho quadrático no caso médio e no pior caso (entradas randômica e decrescente).

4.2 Selection Sort

A análise de complexidade do Selection Sort, considerando as diferentes situações de entrada (crescente, decrescente e randômica), foi realizada manuscrita, com a devida justificativa, contando o número de execuções de cada linha do algoritmo, no mesmo formato adotado no capítulo anterior, conforme exigido pelo enunciado do trabalho.

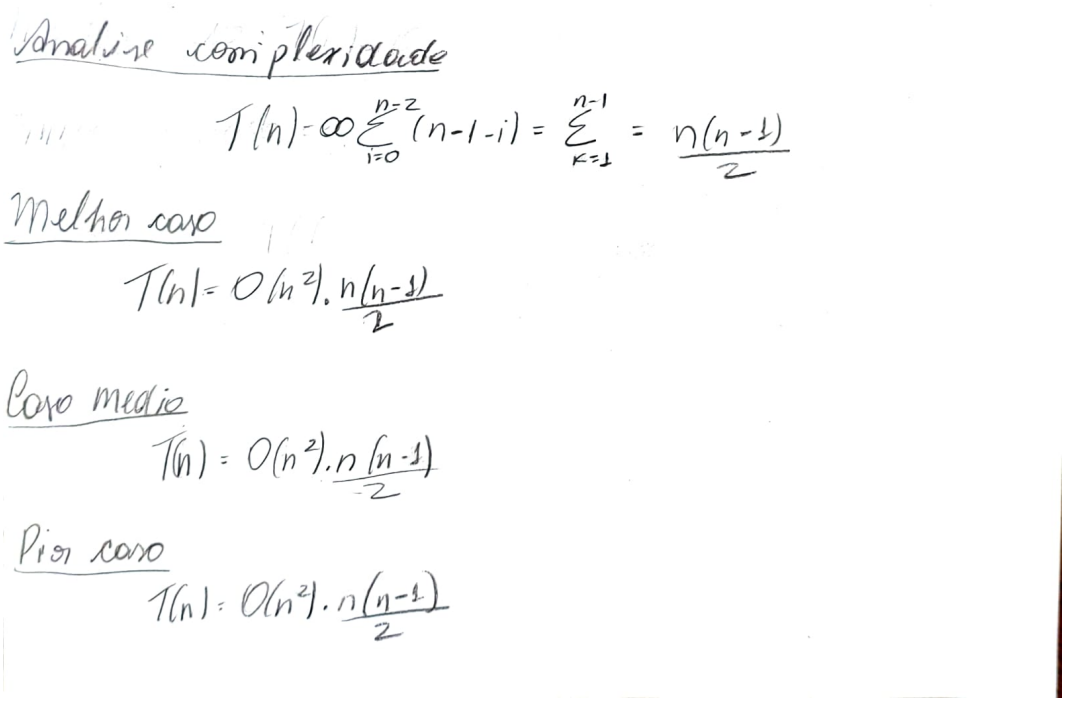


Figura 8: Análise de complexidade manuscrita do algoritmo Selection Sort

Diferentemente do Insertion Sort, o laço interno do Selection Sort sempre percorre integralmente as posições restantes à procura do menor elemento, independentemente da ordem dos dados. Por isso, os três casos coincidem na mesma ordem de grandeza:

Melhor caso (entrada crescente): ainda assim são realizadas $\sum_{i=0}^{n-2}(n-1-i) = \frac{n(n-1)}{2}$ comparações; nenhuma troca é necessária, mas o número de comparações já garante $T(n) = \Theta(n^2)$.

Caso médio (entrada randômica): o número de comparações permanece $\frac{n(n-1)}{2}$; o número esperado de trocas passa a ser da ordem de n (bem menor que o número de comparações), mantendo $T(n) = \Theta(n^2)$.

Pior caso (entrada decrescente): o número de comparações continua $\frac{n(n-1)}{2}$ e o

número de trocas atinge seu máximo, $n - 1$; como esse termo é linear e as comparações são $\Theta(n^2)$, o termo quadrático domina e $T(n) = \Theta(n^2)$.

Ou seja, ao contrário do Insertion Sort e do Bubble Sort, no Selection Sort os três casos (melhor, médio e pior) têm exatamente a mesma ordem de grandeza, $\Theta(n^2)$, pois o número de comparações não depende do tipo de entrada — apenas o número de trocas (sempre $O(n)$, desprezível diante das comparações) varia. Os resultados experimentais apresentados na próxima seção confirmam essa previsão teórica: os tempos medidos para entradas crescente, decrescente e randômica são praticamente idênticos entre si, para um mesmo tamanho de instância.

4.3 Bubble Sort

A análise de complexidade do Bubble Sort, considerando as diferentes situações de entrada (crescente, decrescente e randômica), foi realizada manuscrita, com a devida justificativa, contando o número de execuções de cada linha do algoritmo, no mesmo formato adotado nos capítulos anteriores, conforme exigido pelo enunciado do trabalho.

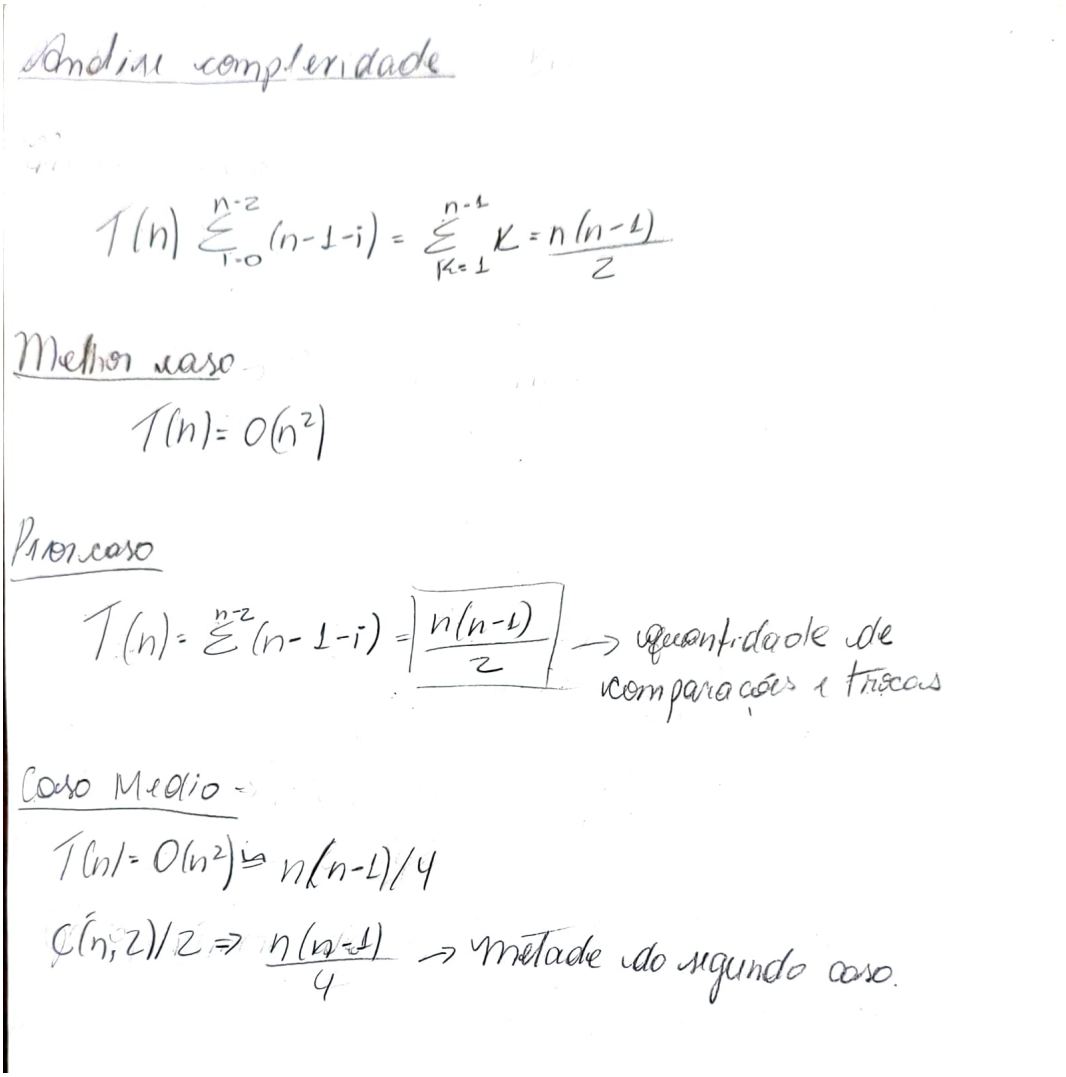


Figura 9: Análise de complexidade manuscrita do algoritmo Bubble Sort

Em resumo, a análise manuscrita chega às seguintes conclusões para cada situação de entrada, considerando a otimização de parada antecipada (*flag* de troca) implementada:

Melhor caso (entrada crescente): a primeira passada não realiza nenhuma troca, o que aciona a parada antecipada; apenas $n - 1$ comparações são feitas, logo $T(n) = O(n)$.

Caso médio (entrada randômica): o número de comparações permanece $\approx \frac{n(n-1)}{2}$ (a parada antecipada raramente ocorre antes da última passada), mas o número esperado de trocas cai para cerca de metade do pior caso, $\frac{n(n-1)}{4}$ — o mesmo raciocínio de inversões esperadas usado no caso médio do Insertion Sort (Capítulo 1). Ainda assim, $T(n) = \Theta(n^2)$.

Pior caso (entrada decrescente): todas as $n - 1$ passadas são necessárias, cada uma com o número máximo de comparações e trocas, totalizando $\frac{n(n-1)}{2}$ comparações e o mesmo número de trocas, ou seja, $T(n) = \Theta(n^2)$.

Os resultados experimentais apresentados na próxima seção deste capítulo são compatíveis com essa análise teórica: desempenho linear no melhor caso (entrada crescente, graças à parada antecipada) e desempenho quadrático no caso médio e no pior caso (entradas randômica e decrescente). Vale destacar que, apesar de Insertion Sort e Bubble Sort compartilharem a mesma ordem de grandeza no pior caso, cada troca do Bubble Sort custa três atribuições, contra apenas uma atribuição por deslocamento no Insertion Sort, o que tende a torná-lo mais lento na prática. Os tempos medidos, porém, mostram que o custo de cada instrução não é o único fator: na entrada randômica o Bubble Sort foi cerca de 11,6 vezes mais lento que o Insertion Sort (938,7 s contra 80,6 s para 1.000.000 de elementos), enquanto na entrada decrescente os dois tiveram desempenho equivalente (155,6 s contra 167,0 s). Na entrada randômica, parte da diferença vem do número de comparações (cerca de $n(n-1)/2$ no Bubble Sort contra $n(n-1)/4$ no Insertion Sort); o restante provavelmente decorre da previsão de desvios do processador: na entrada decrescente a condição `vetor[j] > vetor[j+1]` é sempre verdadeira, um padrão que o processador prevê quase perfeitamente, ao passo que na entrada randômica ela é imprevisível e causa perdas de desempenho do *pipeline*. Essa é uma explicação plausível, mas não foi verificada neste trabalho.

4.4 Shell Sort

Conforme orientado no enunciado do trabalho, a análise do Shell Sort não segue o mesmo roteiro de contagem de instruções aplicado aos demais algoritmos, pois sua complexidade depende diretamente da sequência de intervalos (*gaps*) utilizada. Em vez disso, foi realizada uma pesquisa bibliográfica sobre a complexidade da sequência de gaps efetivamente utilizada na implementação (a sequência original de Shell: tamanho/2, tamanho/4, ..., 1), cujo conteúdo manuscrito é apresentado a seguir.

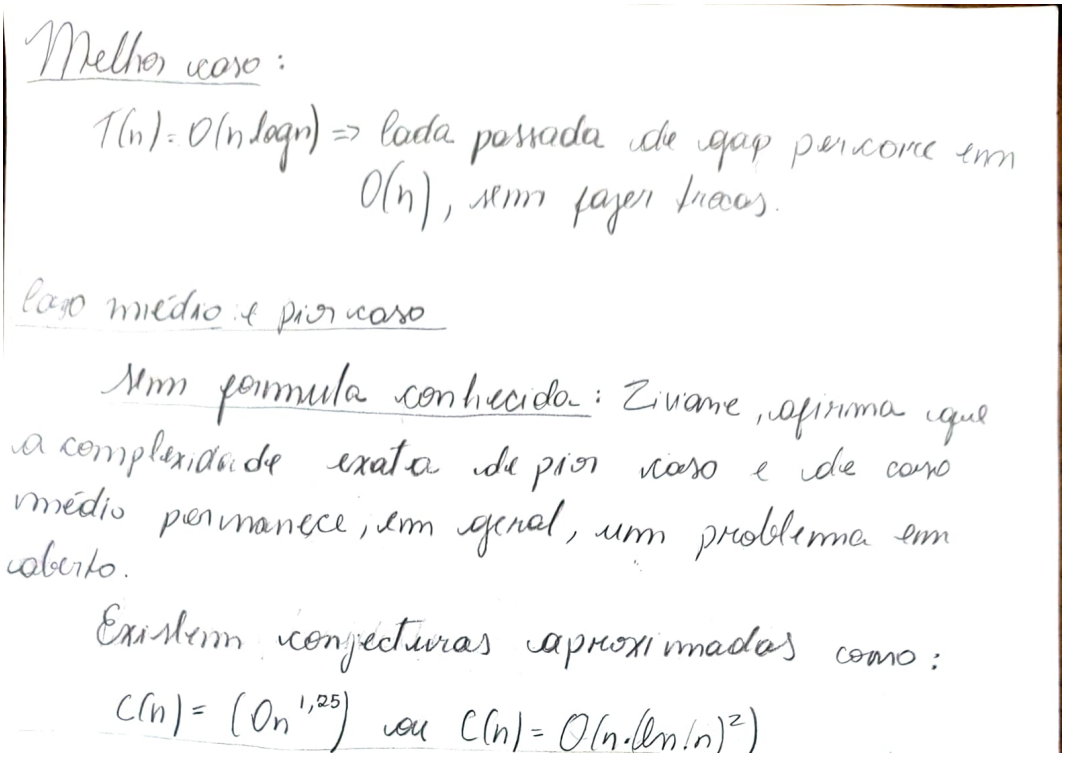


Figura 10: Pesquisa e análise de complexidade manuscrita do algoritmo Shell Sort

A seguir, apresenta-se essa mesma pesquisa em forma digitada, organizada no mesmo formato de melhor/médio/pior caso adotado nos capítulos anteriores e devidamente referenciada, para apoiar a versão manuscrita.

Melhor caso (entrada crescente): $T(n) = O(n \log n)$**.** Cada passada de gap percorre o vetor em $O(n)$ e, como qualquer subsequência de um vetor já ordenado também está ordenada, nenhum deslocamento é necessário em nenhuma passada. Como o número de passadas é $O(\log n)$ (o gap é dividido por 2 a cada passada), o tempo total é $T(n) = O(n \log n)$.

Pior caso: $T(n) = \Theta(n^2)$**.** Donald Knuth demonstrou que a sequência original de Shell (potências de 2) tem pior caso quadrático: existem entradas construídas especificamente para essa sequência de gaps — não a simples entrada decrescente — que forçam $\Theta(n^2)$ comparações e deslocamentos, principalmente quando n é uma potência de 2 (KNUTH, 1998). É por isso que a entrada decrescente testada neste trabalho **não** atinge esse pior caso: qualquer subsequência de um vetor totalmente decrescente também está em ordem decrescente, de modo que cada passada de gap continua limitada a deslocamentos curtos, resultando em comportamento próximo do melhor caso ($O(n \log n)$), e não do pior caso teórico.

Caso médio (entrada randômica): sem fórmula fechada conhecida. Ao contrário dos três algoritmos anteriores, não existe uma dedução simples de caso médio para o Shell Sort, e é justamente esse ponto que justifica o caráter de pesquisa bibliográfica pedido pelo enunciado para este capítulo. Ziviani (2012), ao apresentar o Shellsort, afirma que a complexidade exata de pior caso e de caso médio do algoritmo permanece, em geral, um problema em aberto: nem mesmo para a sequência de incrementos mais estudada na literatura — a de Knuth, $h_s = 3h_{s-1}+1$ (ou seja, $1, 4, 13, 40, 121, \ldots$), recomendada no lugar da sequência original de Shell por ser consideravelmente mais eficiente na prática — existe uma prova fechada de complexidade média; há apenas **conjecturas**, como $C(n) = O(n^{1,25})$ ou $C(n) = O(n \cdot (\ln n)^2)$ para o número de comparações. Se até a melhor sequência de gaps conhecida tem sua complexidade média apenas conjecturada na literatura, é esperado que a sequência mais simples (a original de Shell, usada neste trabalho) também não possua uma fórmula fechada de caso médio — apenas o comportamento empírico observado nos testes, que ficou sensivelmente abaixo de $\Theta(n^2)$ para todos os tamanhos testados.

Em resumo, a escolha da sequência de gaps é o fator decisivo na complexidade do Shell Sort — um ponto reforçado tanto por Knuth (1998), que provou o pior caso quadrático da sequência original, quanto por Ziviani (2012), que recomenda sequências mais sofisticadas justamente por essa sensibilidade à sequência escolhida. Ainda assim, mesmo utilizando a sequência mais simples possível, o Shell Sort superou folgadamente Insertion, Selection e Bubble Sort na prática, para todos os tipos de entrada testados neste trabalho, como mostram os resultados da próxima seção; a comparação com o Merge Sort é feita na Tabela de Comparação e na Conclusão.

4.5 Merge Sort

A análise de complexidade do Merge Sort, considerando as diferentes situações de entrada (crescente, decrescente e randômica), foi realizada manuscrita, com a devida justificativa, contando o número de execuções de cada linha da função de intercalação (associando uma constante de custo c_i a cada linha), escrevendo a recorrência da função recursiva e resolvendo-a, conforme exigido pelo enunciado do trabalho.

Em resumo, a análise manuscrita chega às seguintes conclusões. **Intercalação:** para intercalar um trecho de m elementos, seja p o número de iterações do primeiro laço (que é também o número de comparações) e sejam s e t os elementos que restam, respectivamente, na metade esquerda e na direita quando a outra se esgota. Como cada elemento é copiado exatamente uma vez para o vetor auxiliar, $p+s+t = m$. Somando as execuções de cada linha de `Intercalar`, obtém-se $T_I(m) = (c_2+c_3+c_4)\,p+(c_5+c_6)\,s+(c_7+c_8)\,t+(c_9+c_{10})\,m+K$, com K constante. Como todos os coeficientes são positivos e $p+s+t = m$, resulta $T_I(m) = \Theta(m)$ em qualquer situação: o que varia de uma entrada para outra é apenas como os m elementos se dividem entre p, s e t.

Recorrência: `MergeSortRecursivo` faz duas chamadas sobre metades do vetor e uma intercalação, ou seja, $T(n) = 2\,T(n/2) + T_I(n) + c = 2\,T(n/2) + \Theta(n)$, com $T(1) = \Theta(1)$. Como o vetor pode ser dividido ao meio $\log_2 n$ vezes até restarem trechos de um único elemento, a árvore de recursão tem cerca de $\log_2 n + 1$ níveis e, em cada nível, as intercalações percorrem, no total, os n elementos do vetor, a um custo $\Theta(n)$ por nível. O custo total é, portanto, $\Theta(n \log n)$ — resultado que também decorre do caso 2 do Teorema Mestre (CORMEN et al., 2009), com $a = 2$, $b = 2$ e $f(n) = \Theta(n)$.

Melhor caso (entrada crescente ou decrescente): em cada intercalação, todos os elementos de uma das metades são menores que os da outra; assim, o primeiro laço termina após cerca de $m/2$ comparações e o restante da outra metade é copiado diretamente. O custo $T_I(m)$ continua linear, e a recorrência não muda, de modo que $T(n) = \Theta(n \log n)$, com cerca de $\frac{n}{2}\log_2 n$ comparações no total.

Pior caso: ocorre quando os elementos das duas metades se alternam em todas as intercalações; o primeiro laço só termina após $p = m - 1$ comparações, totalizando $n \log_2 n - n + 1$ comparações (para n potência de 2). Ainda assim, $T(n) = \Theta(n \log n)$: apenas a

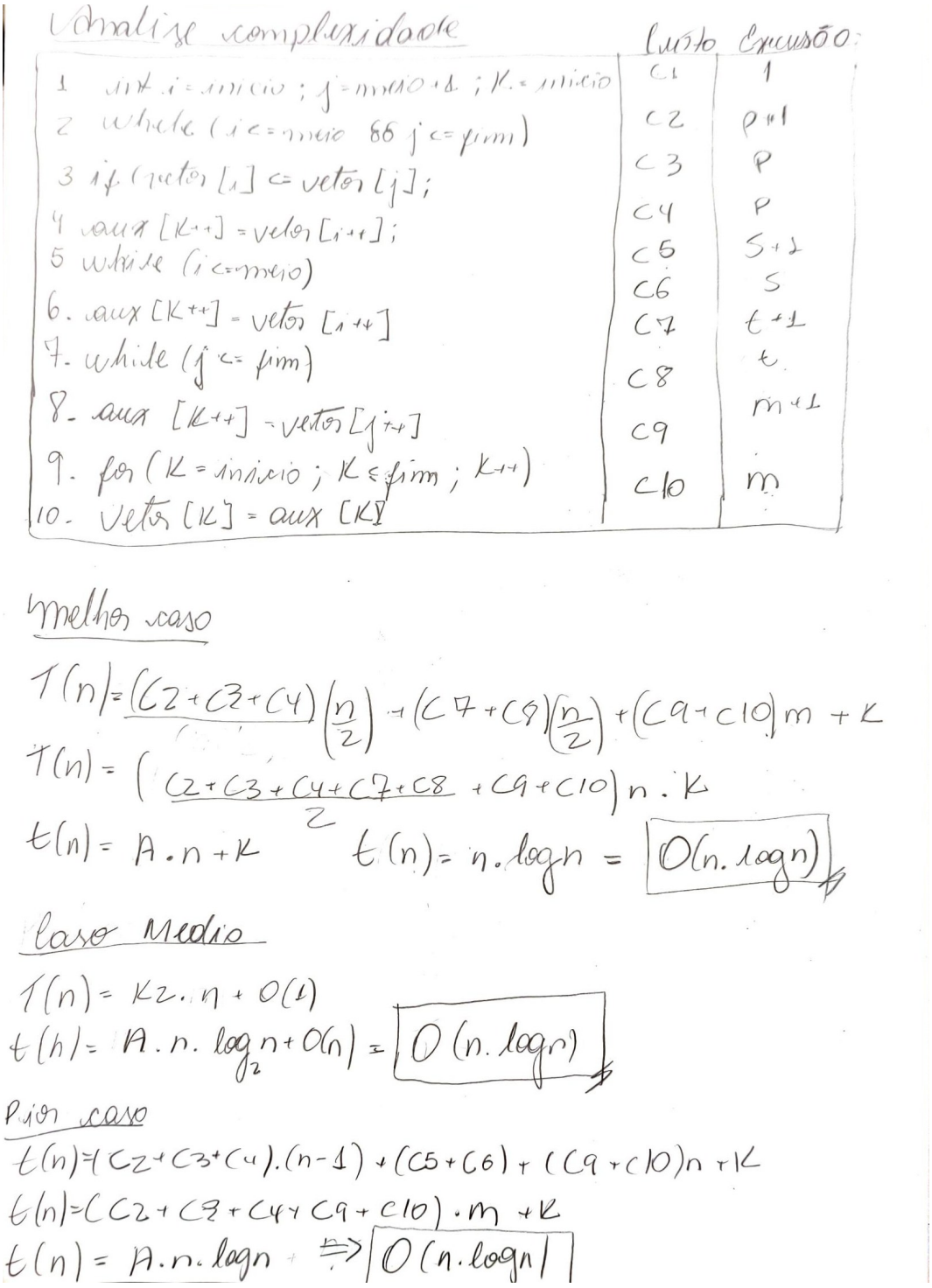


Figura 11: Análise de complexidade manuscrita do algoritmo Merge Sort

constante é maior. Nenhuma das três entradas testadas é exatamente esse pior caso.

Caso médio (entrada randômica): as duas metades se entrelaçam de forma aleatória e, quando uma delas se esgota, restam em média apenas poucos elementos da outra; logo, p fica próximo de $m - 1$ e o número de comparações fica próximo do limite do pior caso, com $T(n) = \Theta(n \log n)$.

Diferentemente do Insertion e do Bubble Sort, **melhor, médio e pior caso têm, portanto, a mesma complexidade**, $\Theta(n \log n)$, pois a divisão e a intercalação são executadas por completo qualquer que seja a ordem dos dados. O número de comparações fica sempre entre $\frac{1}{2} n \log_2 n$ e $n \log_2 n$ (SEDGEWICK; WAYNE, 2011), e o número de cópias de elementos é o mesmo em todos os casos, cerca de $2n \log_2 n$.

Para conferir essa previsão, foi executada uma versão do algoritmo instrumentada para contar as operações (usada apenas para esta verificação, fora do código entregue) sobre 1.000.000 de elementos: foram contadas 10.066.432 comparações para a entrada crescente, 9.884.992 para a decrescente e 18.673.990 para a randômica, valores compatíveis com os limites teóricos de, respectivamente, cerca de 9.965.784 ($\frac{n}{2} \log_2 n$) e 18.931.570 ($n \log_2 n - n + 1$) — as pequenas diferenças decorrem de 1.000.000 não ser uma potência de 2. O número de cópias foi de 39.902.848 nos três casos, ou seja, cerca de $2n \log_2 n$, independentemente da ordem da entrada.

Em contrapartida, o algoritmo exige memória adicional proporcional a n (o vetor auxiliar), enquanto Insertion, Selection, Bubble e Shell Sort ordenam no próprio vetor, com memória adicional constante. Os resultados experimentais apresentados na próxima seção deste capítulo são compatíveis com essa análise: os tempos das três entradas cresceram aproximadamente na proporção $n \log n$, sem a explosão quadrática observada nos três primeiros algoritmos.

5 TABELA E GRÁFICO

5.1 Insertion Sort

A Tabela 1 apresenta os tempos de execução, em segundos, obtidos para o Insertion Sort nos seis tamanhos de instância exigidos pelo trabalho, para cada um dos três tipos de entrada testados.

	10	100	1.000	10.000	100.000	1.000.000
Crescente	0,000010	0,000001	0,000004	0,000015	0,000074	0,000584
Decrescente	0,000007	0,000006	0,000174	0,015947	1,620129	166,979744
Random	0,000002	0,000002	0,000096	0,008208	0,844211	80,586551

Tabela 1: Tempo de execução (em segundos) do Insertion Sort por tamanho e tipo de instância

O Gráfico 12 apresenta esses mesmos dados em escala linear. Nele fica evidente que, para tamanhos de até 10.000 elementos, os tempos de execução são praticamente imperceptíveis nas três situações. A diferença de comportamento só se torna visualmente clara a partir de 100.000 elementos, quando o tempo das entradas decrescente e randômica cresce de forma acentuada, enquanto a entrada crescente permanece próxima de zero.

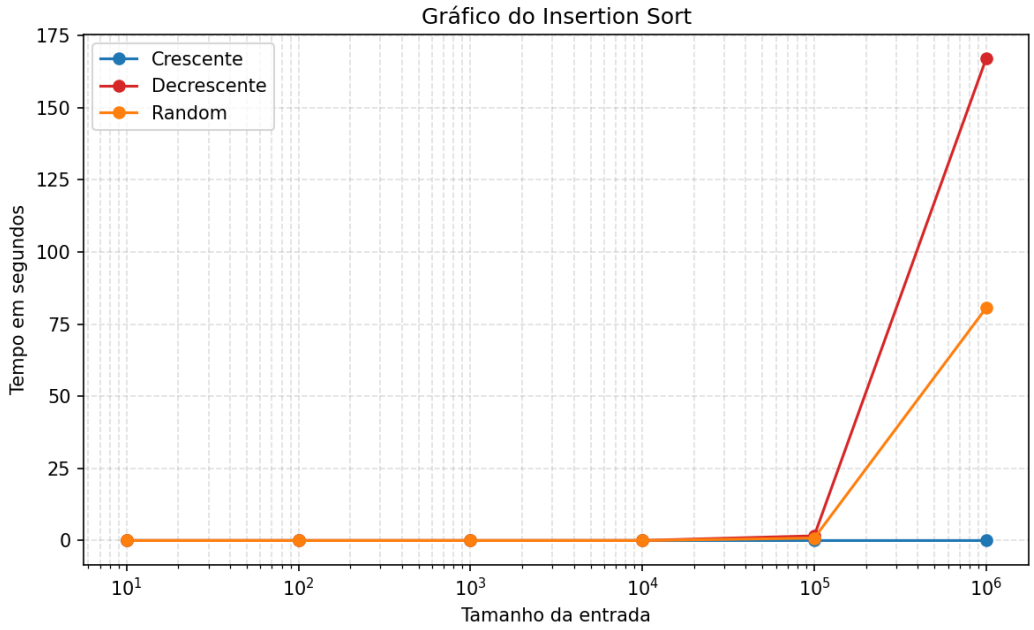


Figura 12: Gráfico de tempo por tamanho do algoritmo Insertion Sort (escala linear)

Como o crescimento quadrático faz com que os valores nas menores instâncias fiquem visualmente esmagados no gráfico em escala linear, o Gráfico 13 apresenta os mesmos dados em escala logarítmica no eixo do tempo, o que permite observar o comportamento do algoritmo em todos os tamanhos testados, inclusive os menores.

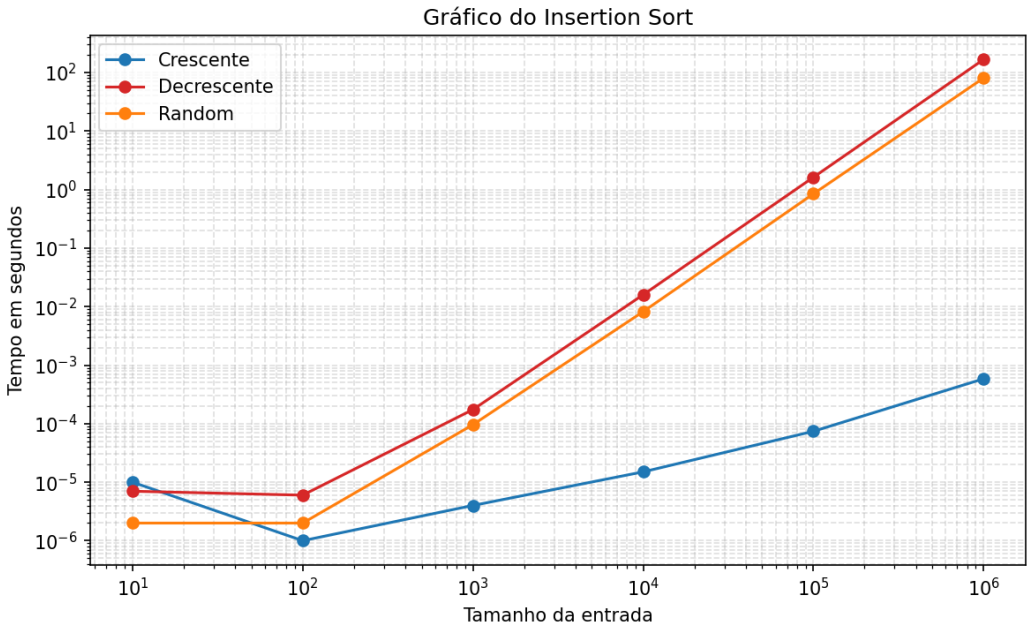


Figura 13: Gráfico de tempo por tamanho do algoritmo Insertion Sort (escala logarítmica)

Analisando os resultados, observa-se que a entrada crescente apresenta desempenho linear: mesmo para 1.000.000 de elementos, o tempo de execução foi inferior a um milésimo de segundo (cerca de 0,0006 segundo), já que o laço interno de deslocamento praticamente não é executado quando o vetor já está ordenado. Já a entrada decrescente, que representa o pior caso do algoritmo, teve o tempo de execução crescendo de forma quadrática, chegando a aproximadamente 167,0 segundos para 1.000.000 de elementos. A entrada randômica, representando o caso médio, também cresceu de forma quadrática, porém com tempo de execução um pouco inferior à metade do pior caso (cerca de 80,6 segundos para 1.000.000 de elementos, ou 48% do tempo da entrada decrescente), o que é compatível com o número esperado de comparações e deslocamentos em uma entrada aleatória, que é metade do observado no pior caso.

5.2 Selection Sort

A Tabela 2 apresenta os tempos de execução, em segundos, obtidos para o Selection Sort nos seis tamanhos de instância exigidos pelo trabalho, para cada um dos três tipos de entrada testados.

	10	100	1.000	10.000	100.000	1.000.000
Crescente	0,000002	0,000007	0,000896	0,093473	9,351039	934,897502
Decrescente	0,000002	0,000008	0,000893	0,092843	9,336395	937,348384
Random	0,000003	0,000010	0,000903	0,092938	9,362431	932,549777

Tabela 2: Tempo de execução (em segundos) do Selection Sort por tamanho e tipo de instância

O Gráfico 14 apresenta esses mesmos dados em escala linear. Diferentemente do Insertion Sort, aqui as três curvas ficam praticamente sobrepostas em todos os tamanhos, o que já era esperado pela análise de complexidade: o número de comparações do Selection Sort não depende do tipo de entrada. Para 1.000.000 de elementos, por exemplo, os três tempos ficaram entre 932,5 s e 937,3 s, uma diferença de apenas 0,5% entre o menor e o maior.

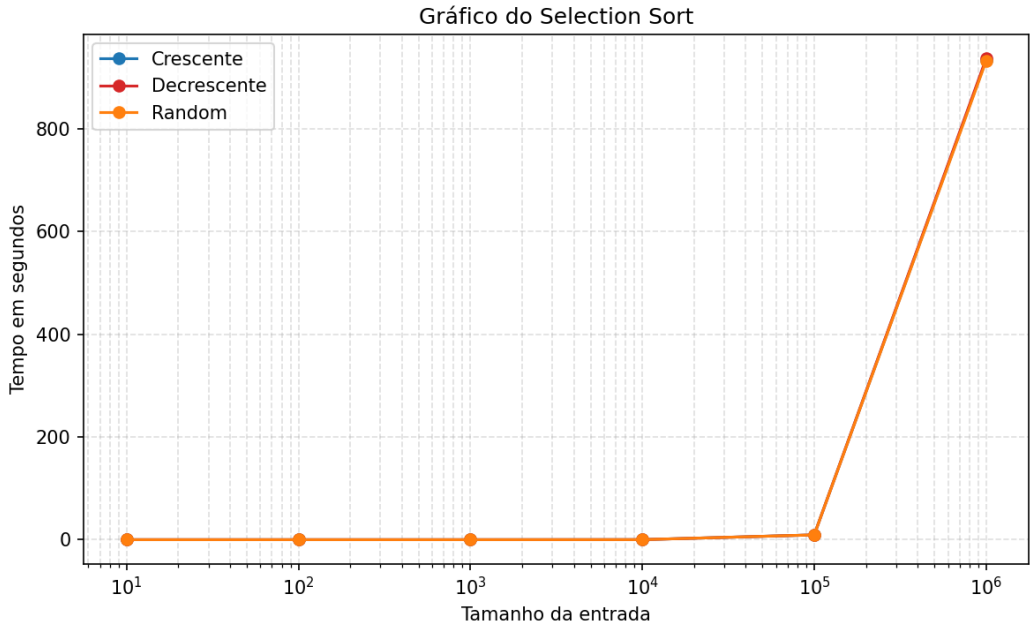


Figura 14: Gráfico de tempo por tamanho do algoritmo Selection Sort (escala linear)

Como as três curvas são muito próximas, o Gráfico 15 em escala logarítmica ajuda a confirmar que, mesmo em pequena escala, os tempos de Crescente, Decrescente e Random

seguem lado a lado durante todo o intervalo de tamanhos testado – a assinatura característica de um algoritmo cujo número de comparações independe da ordem dos dados de entrada, conforme discutido na análise de complexidade deste capítulo.

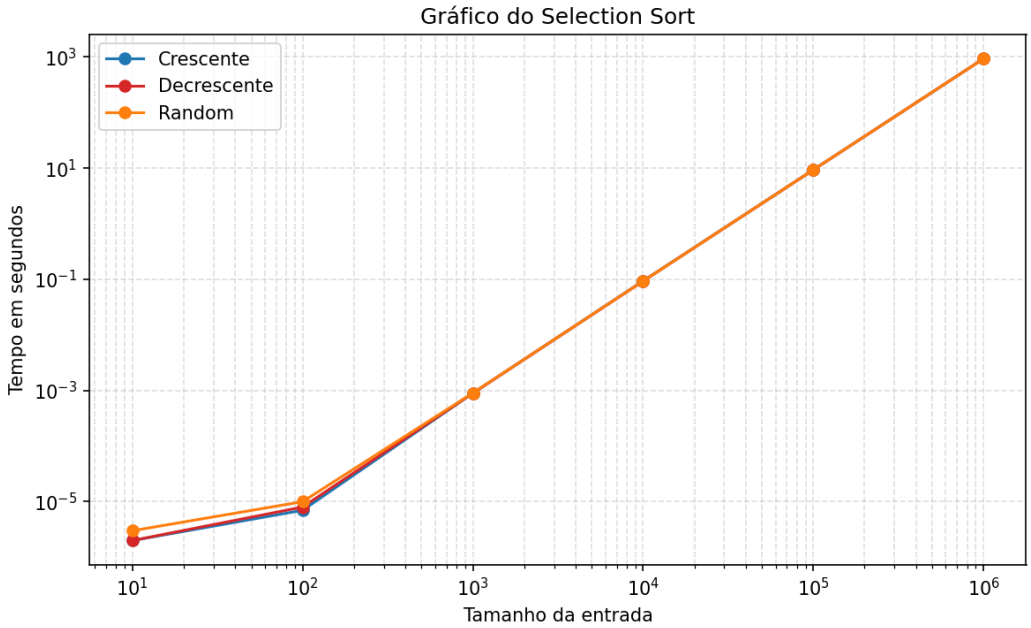


Figura 15: Gráfico de tempo por tamanho do algoritmo Selection Sort (escala logarítmica)

5.3 Bubble Sort

A Tabela 3 apresenta os tempos de execução, em segundos, obtidos para o Bubble Sort nos seis tamanhos de instância exigidos pelo trabalho, para cada um dos três tipos de entrada testados.

	10	100	1.000	10.000	100.000	1.000.000
Crescente	0,000001	0,000001	0,000000	0,000003	0,000033	0,000321
Decrescente	0,000002	0,000002	0,000166	0,015925	1,583171	155,625005
Random	0,000003	0,000008	0,000326	0,027070	8,899341	938,659248

Tabela 3: Tempo de execução (em segundos) do Bubble Sort por tamanho e tipo de instância

O Gráfico 16 apresenta esses mesmos dados em escala linear. Assim como no Insertion Sort, a entrada crescente permanece praticamente em zero em todos os tamanhos (graças à parada antecipada do algoritmo), enquanto as entradas decrescente e randômica crescem de forma acentuada a partir de 100.000 elementos. Diferentemente do Insertion Sort, porém, aqui a entrada randômica é a mais lenta das três: para 1.000.000 de elementos, 938,7 s, contra 155,6 s da entrada decrescente.

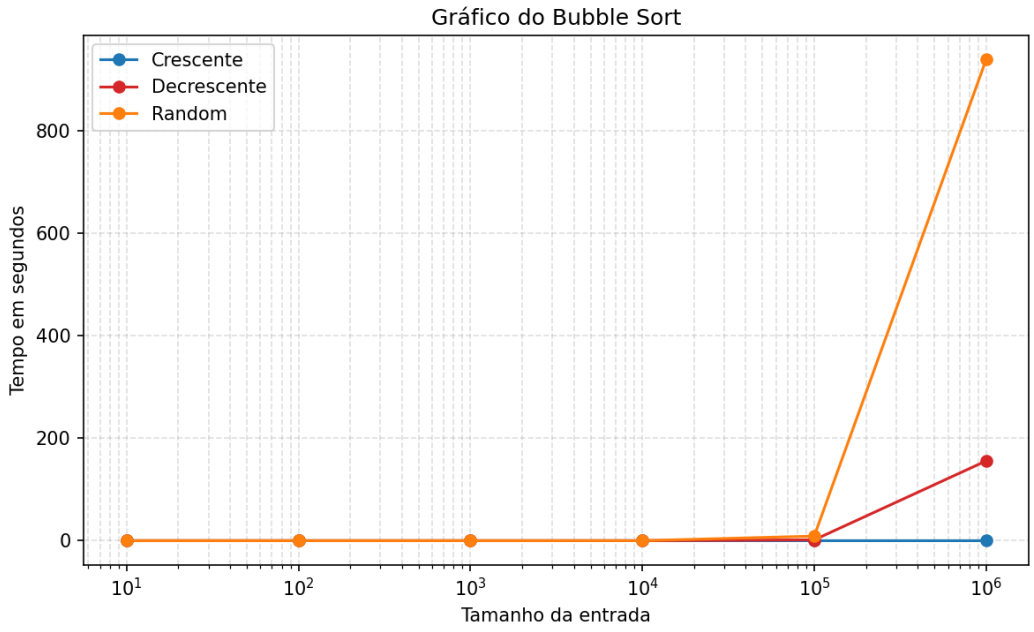


Figura 16: Gráfico de tempo por tamanho do algoritmo Bubble Sort (escala linear)

O Gráfico 17 em escala logarítmica confirma o comportamento linear da entrada crescente

e o comportamento quadrático das entradas decrescente e randômica. O tempo de 0,000000 s medido para 1.000 elementos na entrada crescente (abaixo da resolução do relógio, de cerca de 1 μs) não pode ser representado em escala logarítmica, e por isso a curva da entrada crescente apresenta uma interrupção nesse ponto.

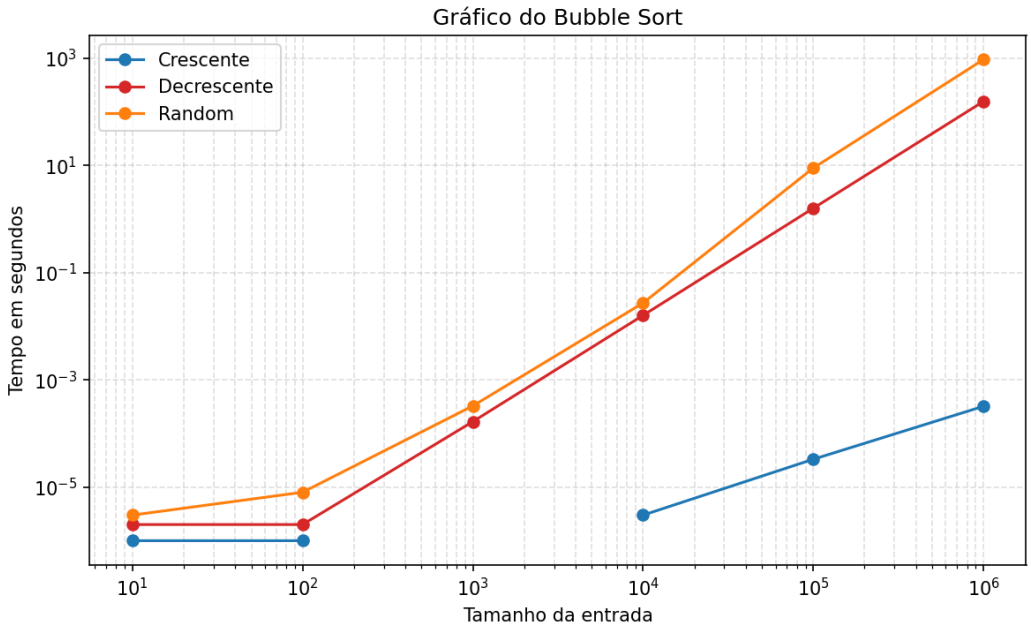


Figura 17: Gráfico de tempo por tamanho do algoritmo Bubble Sort (escala logarítmica)

Comparando com o Insertion Sort (Capítulo 1), nota-se que, na entrada decrescente, os dois algoritmos tiveram desempenho equivalente (155,6 s para o Bubble Sort contra 167,0 s para o Insertion Sort, para 1.000.000 de elementos), enquanto na entrada randômica o Bubble Sort foi cerca de 11,6 vezes mais lento (938,7 s contra 80,6 s). Parte dessa diferença é explicada pelo número de comparações: em uma entrada aleatória o Bubble Sort realiza cerca de $n(n-1)/2$ comparações, o dobro das cerca de $n(n-1)/4$ do Insertion Sort. A diferença restante provavelmente decorre de efeitos de hardware, discutidos na análise de complexidade deste capítulo, que a simples contagem de operações não captura.

5.4 Shell Sort

A Tabela 4 apresenta os tempos de execução, em segundos, obtidos para o Shell Sort nos seis tamanhos de instância exigidos pelo trabalho, para cada um dos três tipos de entrada testados.

	10	100	1.000	10.000	100.000	1.000.000
Crescente	0,000003	0,000001	0,000005	0,000065	0,000690	0,008251
Decrescente	0,000002	0,000002	0,000009	0,000083	0,001082	0,011790
Random	0,000003	0,000004	0,000052	0,000655	0,008226	0,106033

Tabela 4: Tempo de execução (em segundos) do Shell Sort por tamanho e tipo de instância

O Gráfico 18 apresenta esses mesmos dados em escala linear. Diferentemente dos três algoritmos anteriores, mesmo a entrada decrescente permanece com tempo de execução muito baixo – como discutido na análise de complexidade, uma entrada totalmente decrescente não é o pior caso teórico da sequência de gaps utilizada. Para 1.000.000 de elementos, os tempos foram de 0,008 s (crescente), 0,012 s (decrescente) e 0,106 s (randômica).

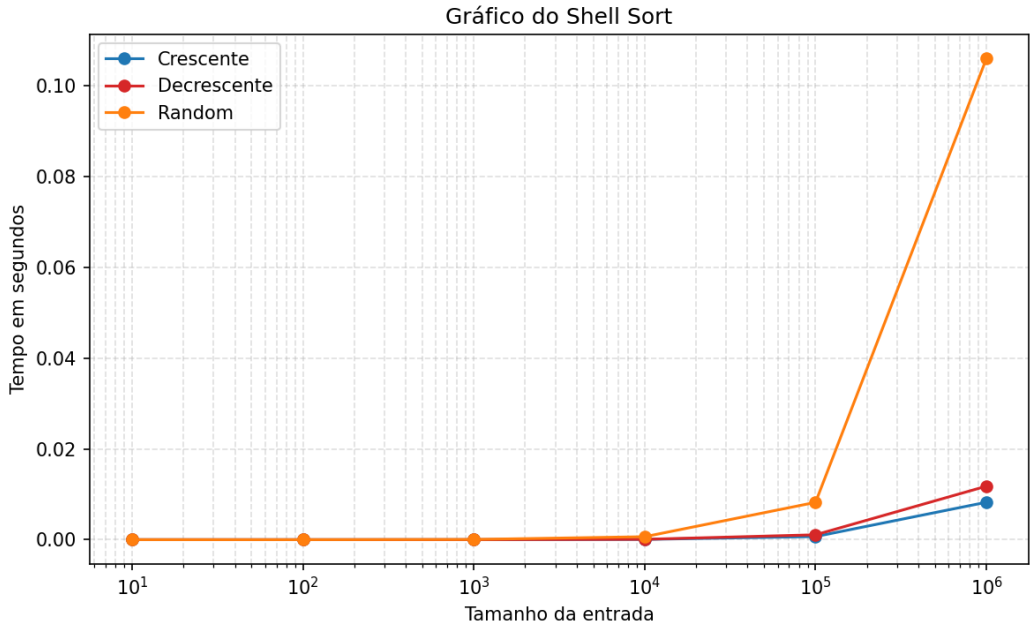


Figura 18: Gráfico de tempo por tamanho do algoritmo Shell Sort (escala linear)

O Gráfico 19 em escala logarítmica evidencia que mesmo a entrada randômica, a mais lenta das três para este algoritmo, é ordens de grandeza mais rápida do que qualquer um

dos três algoritmos anteriores ($O(n^2)$) para o mesmo tamanho de instância: para 1.000.000 de elementos randômicos, cerca de 760 vezes mais rápida que o Insertion Sort e cerca de 8.800 vezes mais rápida que o Selection e o Bubble Sort. Ao aumentar o tamanho de 100.000 para 1.000.000 de elementos (10 vezes), o tempo foi multiplicado por 12,0 (crescente), 10,9 (decrescente) e 12,9 (randômica), valores próximos do fator 12,0 esperado para $n \log n$ e muito abaixo do fator 100 de um crescimento quadrático. Essa é a principal conclusão prática deste capítulo, retomada na Tabela de Comparação e no Gráfico Geral ao final deste trabalho, onde o Shell Sort também é comparado com o Merge Sort.

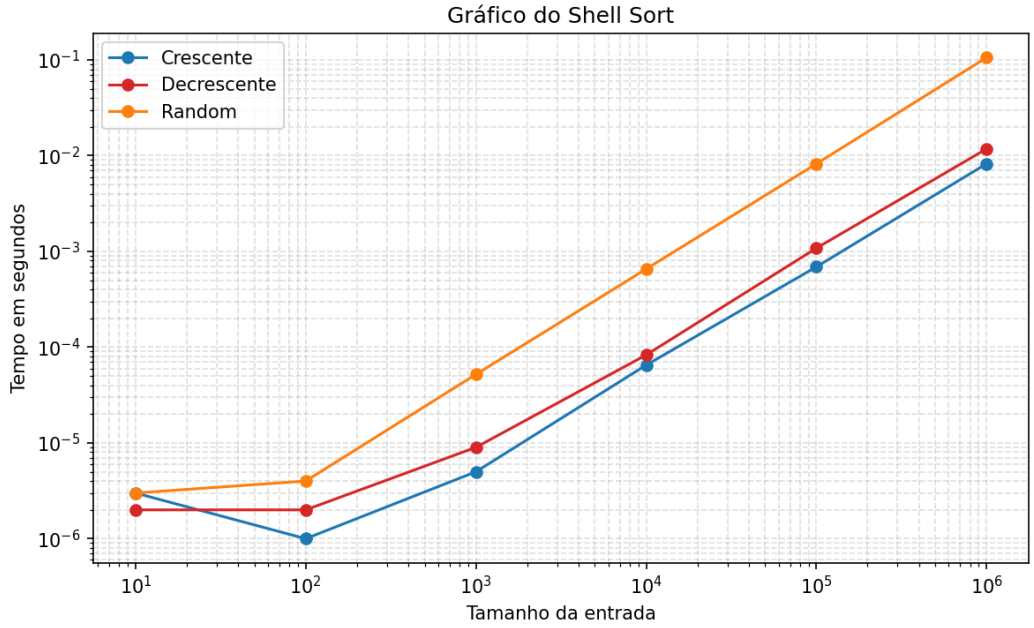


Figura 19: Gráfico de tempo por tamanho do algoritmo Shell Sort (escala logarítmica)

5.5 Merge Sort

A Tabela 5 apresenta os tempos de execução, em segundos, obtidos para o Merge Sort nos seis tamanhos de instância exigidos pelo trabalho, para cada um dos três tipos de entrada testados.

	10	100	1.000	10.000	100.000	1.000.000
Crescente	0,000002	0,000002	0,000015	0,000194	0,002070	0,021293
Decrescente	0,000002	0,000002	0,000014	0,000175	0,001809	0,020753
Random	0,000002	0,000009	0,000058	0,000551	0,006542	0,077718

Tabela 5: Tempo de execução (em segundos) do Merge Sort por tamanho e tipo de instância

O Gráfico 20 apresenta esses mesmos dados em escala linear. Os tempos são todos muito pequenos: mesmo para 1.000.000 de elementos o Merge Sort levou menos de 0,08 segundo nas três situações, uma escala milhares de vezes menor que a dos gráficos dos três primeiros algoritmos. As curvas das entradas crescente e decrescente praticamente se sobrepõem, como previsto na análise de complexidade, e ficam abaixo da curva da entrada randômica.

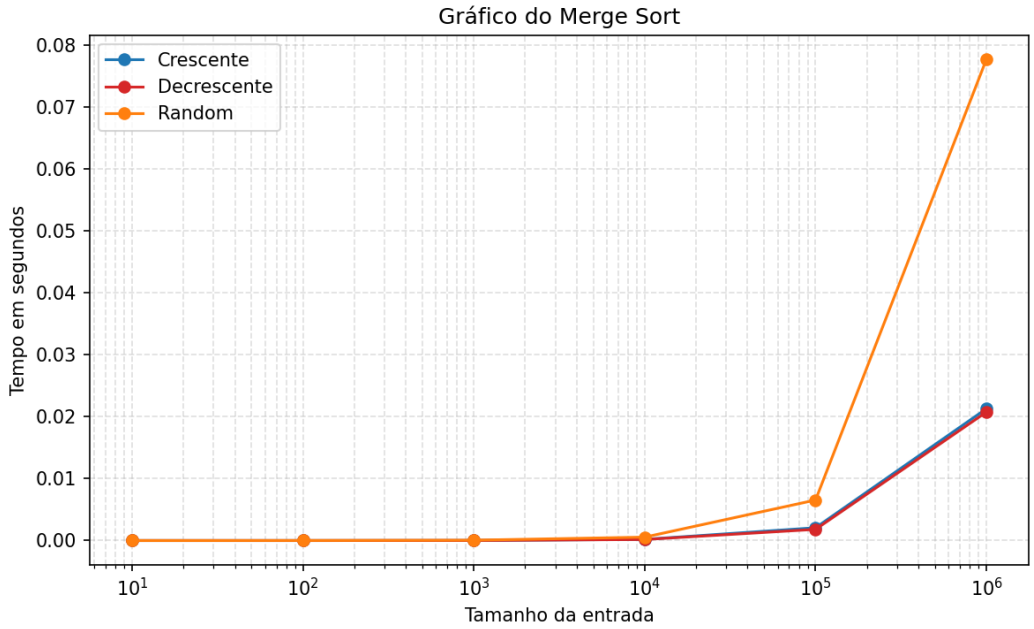


Figura 20: Gráfico de tempo por tamanho do algoritmo Merge Sort (escala linear)

O Gráfico 21 apresenta os mesmos dados em escala logarítmica nos dois eixos. Nessa escala, um crescimento quadrático aparece como reta de inclinação 2, e um crescimento

da ordem de $n \log n$ como reta de inclinação ligeiramente superior a 1: as três curvas do Merge Sort são aproximadamente paralelas entre si e muito menos inclinadas que as dos algoritmos quadráticos. Quantitativamente, ao aumentar o tamanho da entrada de 100.000 para 1.000.000 de elementos (10 vezes), o tempo foi multiplicado por 10,3 (crescente), 11,5 (decrescente) e 11,9 (randômica), valores próximos de 12,0, que é o fator esperado para $n \log n$ nesse intervalo, e muito distantes de 100, que seria o fator esperado em um crescimento quadrático.

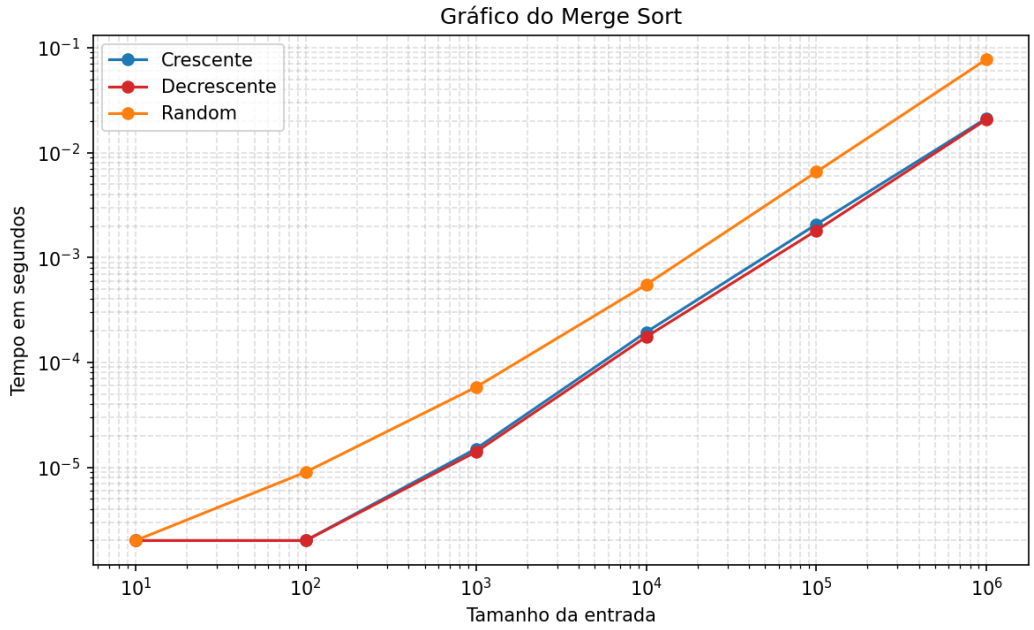


Figura 21: Gráfico de tempo por tamanho do algoritmo Merge Sort (escala logarítmica)

A entrada randômica foi a mais lenta: para 1.000.000 de elementos, 0,077718 s, cerca de 3,7 vezes o tempo das entradas ordenadas. Isso não altera a ordem de complexidade, mas merece uma explicação: a entrada randômica exige quase o dobro de comparações (18,7 milhões contra cerca de 10 milhões) e o mesmo número de cópias (39,9 milhões), o que por si só não explicaria uma diferença de 3,7 vezes. Uma explicação plausível, não verificada neste trabalho, é a previsão de desvios do processador: nas entradas ordenadas o resultado de `vetor[i] <= vetor[j]` segue um padrão regular, que o processador aprende a prever, enquanto na entrada randômica ele é imprevisível e causa perdas de desempenho que a contagem de operações não captura.

6 TABELA DE COMPARAÇÃO

As Tabelas 6, 7 e 8 apresentam, lado a lado, os tempos de execução (em segundos) obtidos pelos cinco algoritmos estudados neste trabalho, para os seis tamanhos de instância exigidos, uma tabela para cada um dos três tipos de entrada testados (Crescente, Decrescente e Random). Todos os valores são os registrados nos arquivos de tempo gerados pelo programa para cada algoritmo.

	10	100	1.000	10.000	100.000	1.000.000
Insertion Sort	0,000010	0,000001	0,000004	0,000015	0,000074	0,000584
Selection Sort	0,000002	0,000007	0,000896	0,093473	9,351039	934,897502
Bubble Sort	0,000001	0,000001	0,000000	0,000003	0,000033	0,000321
Shell Sort	0,000003	0,000001	0,000005	0,000065	0,000690	0,008251
Merge Sort	0,000002	0,000002	0,000015	0,000194	0,002070	0,021293

Tabela 6: Tempo de execução (em segundos) dos 5 algoritmos – entrada Crescente

	10	100	1.000	10.000	100.000	1.000.000
Insertion Sort	0,000007	0,000006	0,000174	0,015947	1,620129	166,979744
Selection Sort	0,000002	0,000008	0,000893	0,092843	9,336395	937,348384
Bubble Sort	0,000002	0,000002	0,000166	0,015925	1,583171	155,625005
Shell Sort	0,000002	0,000002	0,000009	0,000083	0,001082	0,011790
Merge Sort	0,000002	0,000002	0,000014	0,000175	0,001809	0,020753

Tabela 7: Tempo de execução (em segundos) dos 5 algoritmos – entrada Decrescente

	10	100	1.000	10.000	100.000	1.000.000
Insertion Sort	0,000002	0,000002	0,000096	0,008208	0,844211	80,586551
Selection Sort	0,000003	0,000010	0,000903	0,092938	9,362431	932,549777
Bubble Sort	0,000003	0,000008	0,000326	0,027070	8,899341	938,659248
Shell Sort	0,000003	0,000004	0,000052	0,000655	0,008226	0,106033
Merge Sort	0,000002	0,000009	0,000058	0,000551	0,006542	0,077718

Tabela 8: Tempo de execução (em segundos) dos 5 algoritmos – entrada Random

Para 1.000.000 de elementos, os destaques de cada tabela são: na entrada crescente, o Bubble Sort (0,000321 s) e o Insertion Sort (0,000584 s) foram os mais rápidos, por serem lineares nesse cenário, seguidos pelo Shell Sort (0,008251 s) e pelo Merge Sort (0,021293 s), enquanto o Selection Sort levou 934,9 s; na entrada decrescente, o Shell Sort (0,011790 s) e o

Merge Sort (0,020753 s) ficaram quatro ordens de grandeza abaixo dos algoritmos quadráticos, entre os quais Bubble Sort (155,6 s) e Insertion Sort (167,0 s) tiveram desempenho equivalente e o Selection Sort foi o mais lento (937,3 s); na entrada randômica, o Merge Sort foi o mais rápido (0,077718 s), seguido pelo Shell Sort (0,106033 s), muito à frente do Insertion Sort (80,6 s), do Selection Sort (932,5 s) e do Bubble Sort (938,7 s).

7 GRÁFICO GERAL

Os Gráficos 22 e 23 apresentam, respectivamente, o tempo de execução dos cinco algoritmos estudados neste trabalho para entradas já ordenadas (crescente) e para entradas em ordem decrescente (o cenário mais desfavorável para o Insertion e o Bubble Sort), permitindo comparar diretamente o desempenho de Insertion Sort, Selection Sort, Bubble Sort, Shell Sort e Merge Sort. Ambos utilizam escala logarítmica nos dois eixos, necessária porque os tempos variam de microssegundos a centenas de segundos. Valores iguais a zero (tempos menores que a resolução do relógio, de cerca de 1 μs nestas medições) não podem ser representados em escala logarítmica; o único caso neste trabalho é o do Bubble Sort com 1.000 elementos em entrada crescente, que por isso aparece como uma interrupção na curva do Bubble Sort no Gráfico 22.

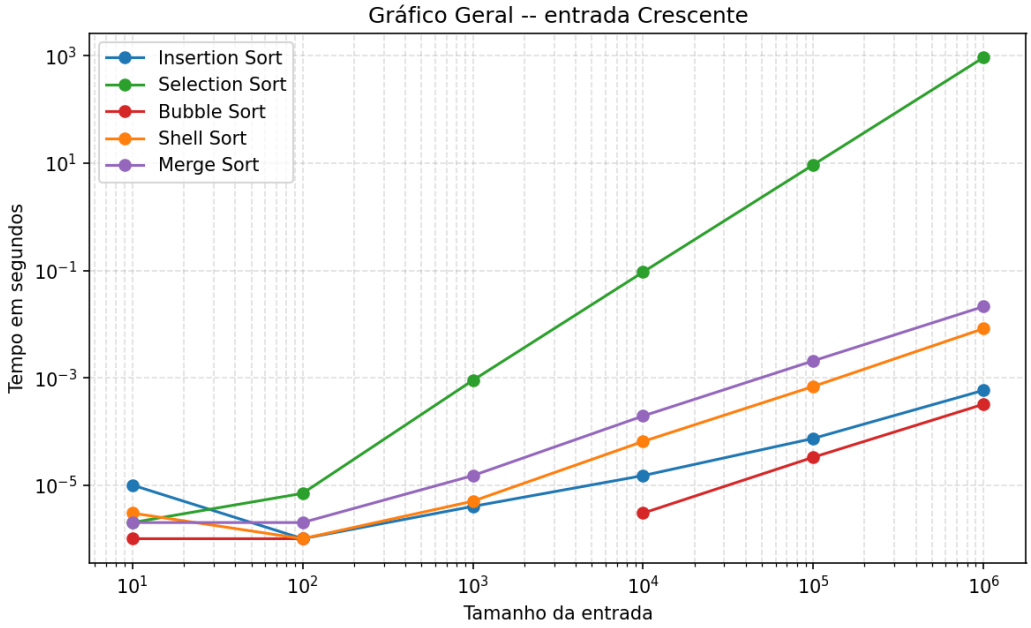


Figura 22: Gráfico geral (escala logarítmica) – entrada crescente

Na entrada crescente (Gráfico 22) os algoritmos se separam em três grupos: o Insertion e o Bubble Sort, que têm desempenho linear nesse cenário e levaram menos de um milésimo de segundo para 1.000.000 de elementos; o Shell Sort e o Merge Sort, da ordem de $n \log n$, com 0,008 s e 0,021 s; e o Selection Sort, que continua quadrático (934,9 s) por não se beneficiar da ordenação prévia. Na entrada decrescente (Gráfico 23) a separação é em

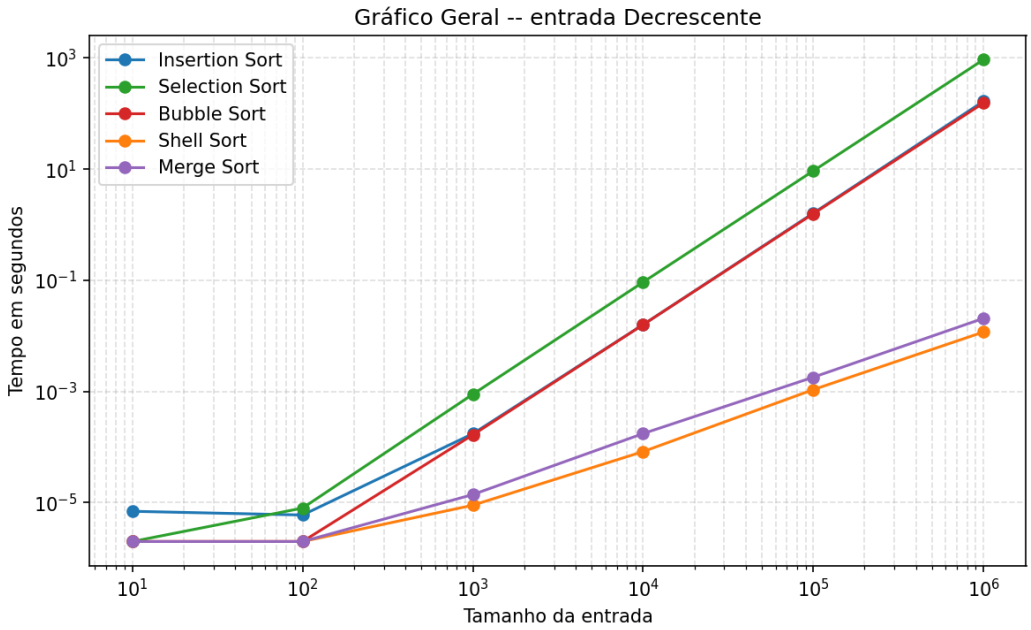


Figura 23: Gráfico geral (escala logarítmica) – entrada decrescente

dois grupos: os três algoritmos quadráticos (Selection Sort com 937,3 s, e Insertion e Bubble Sort com cerca de 160 s, com curvas praticamente sobrepostas) e, cerca de quatro ordens de grandeza abaixo, o Shell Sort (0,012 s) e o Merge Sort (0,021 s).

8 CONCLUSÃO

A partir da implementação e dos experimentos realizados com os cinco algoritmos, foi possível observar, na prática, diferenças de comportamento que confirmam a análise teórica de cada um. Os tempos citados a seguir referem-se a instâncias de 1.000.000 de elementos.

O **Insertion Sort** e o **Bubble Sort** compartilham uma característica importante: ambos são sensíveis ao grau de ordenação da entrada, apresentando desempenho linear ($O(n)$) quando os dados já estão ordenados (0,0006 s e 0,0003 s, respectivamente) e desempenho quadrático ($O(n^2)$) nos casos médio e pior (entradas randômica e decrescente). Entre os dois, o resultado depende do tipo de entrada: com a entrada decrescente os tempos foram equivalentes (167,0 s para o Insertion Sort e 155,6 s para o Bubble Sort), mas com a entrada randômica o Bubble Sort foi cerca de 11,6 vezes mais lento (938,7 s contra 80,6 s). Parte dessa diferença decorre do dobro de comparações do Bubble Sort em uma entrada aleatória ($n(n-1)/2$ contra $n(n-1)/4$) e, provavelmente, de efeitos de hardware que a contagem de operações não captura.

O **Selection Sort** se destacou por um comportamento distinto: sua complexidade é $\Theta(n^2)$ em *todos* os casos (melhor, médio e pior), já que o número de comparações não depende da ordem dos dados — só o de trocas varia, mas é sempre $O(n)$, irrelevante diante das $O(n^2)$ comparações. Isso foi confirmado experimentalmente: os tempos medidos para as entradas crescente, decrescente e randômica ficaram praticamente idênticos entre si (932,5 s a 937,3 s, uma variação de 0,5%). Por não se beneficiar de entradas ordenadas, foi o mais lento na entrada crescente (934,9 s, contra 0,0006 s do Insertion Sort) e equivalente ao Bubble Sort na randômica. Sua vantagem é o número previsível e mínimo de trocas (no máximo $n-1$), útil quando a operação de escrita é cara.

O **Shell Sort** apresentou desempenho muito superior ao dos três algoritmos anteriores em todos os tipos de entrada testados (0,008 s, 0,012 s e 0,106 s para as entradas crescente, decrescente e randômica), mesmo utilizando a sequência de intervalos mais simples (a original de Shell). Embora essa sequência tenha pior caso teórico $\Theta(n^2)$, ele exige uma disposição de dados construída especificamente para isso, o que não ocorre nem com uma entrada totalmente decrescente — por isso as entradas crescente e decrescente exibiram

comportamento próximo de $O(n \log n)$, e a randômica (sem fórmula fechada simples de caso médio) foi cerca de 760 vezes mais rápida que o Insertion Sort.

O **Merge Sort** tem complexidade $\Theta(n \log n)$ em todos os casos, independentemente da ordem inicial dos dados, e foi o único algoritmo cujo tempo não apresentou grande variação com o tipo de entrada: 0,021 s (crescente), 0,021 s (decrescente) e 0,078 s (randômica). Para a entrada randômica foi o mais rápido dos cinco (0,078 s, contra 0,106 s do Shell Sort e 80,6 s do Insertion Sort, o melhor dos três algoritmos quadráticos nesse cenário, ou seja, cerca de mil vezes mais rápido). Quando a entrada já é crescente ou decrescente, o Shell Sort foi mais rápido que o Merge Sort (0,008 s e 0,012 s contra 0,021 s), e o Insertion e o Bubble Sort, lineares na entrada crescente, ainda mais. O preço do Merge Sort é a memória adicional proporcional ao tamanho da entrada (um vetor auxiliar de cerca de 4 MB para 1.000.000 de inteiros), enquanto os outros quatro algoritmos ordenam no próprio vetor.

Em resumo, os experimentos evidenciam que a escolha do algoritmo importa mais do que qualquer ajuste de detalhe: para 1.000.000 de elementos randômicos, os algoritmos de menor complexidade (Merge e Shell Sort) levaram cerca de 0,1 s, contra 80 a 940 s dos três algoritmos $O(n^2)$ — uma diferença de três a quatro ordens de grandeza. Entre os algoritmos $O(n^2)$, o Insertion Sort foi o mais regular: nunca foi o pior em nenhum cenário e foi, de longe, o melhor na entrada randômica, sendo preferível quando se espera entradas quase ordenadas; o Selection Sort tem a vantagem de um número mínimo de trocas, e o Bubble Sort, apesar de didático, não apresentou vantagem prática clara sobre os demais, a não ser na entrada crescente, onde foi o mais rápido por uma margem muito pequena. O Merge Sort é a escolha mais segura quando se precisa de desempenho garantido para qualquer entrada e há memória disponível; o Shell Sort, que mostra como uma modificação simples do Insertion Sort (intervalos decrescentes) produz um ganho expressivo, é uma boa alternativa quando a memória é restrita ou os dados tendem a estar parcialmente ordenados.

Cada instância foi executada uma única vez, com instâncias aleatórias distintas por algoritmo, e medida com `clock()`, de resolução limitada; por isso, diferenças pequenas, como a de 7% entre o Insertion e o Bubble Sort na entrada decrescente, não indicam superioridade de um algoritmo, ao contrário das diferenças de ordens de grandeza, que são robustas.

9 REFERÊNCIAS BIBLIOGRÁFICAS

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.

SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.

KNUTH, Donald E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Boston: Addison-Wesley, 1998.

SHELL, Donald L. A High-Speed Sorting Procedure. *Communications of the ACM*, v. 2, n. 7, p. 30–32, 1959.

ZIVIANI, Nivio. *Projeto de Algoritmos: com Implementações em Java e C++*. São Paulo: Cengage Learning, 2012.

10 Função para calcular o tempo

A seguir apresenta-se a função responsável por orquestrar, para cada instância, a geração do vetor, a execução do algoritmo de ordenação selecionado (recebido como um ponteiro de função, o que permite reutilizar a mesma rotina para os cinco algoritmos deste trabalho) com a respectiva medição de tempo, e o registro dos arquivos de entrada, saída e tempo dentro da pasta correspondente ao algoritmo — pasta essa que também é criada automaticamente pelo próprio programa, caso ainda não exista (arquivos `algoritmos.h` e `main.cpp`). Por brevidade, foram omitidas as mensagens de erro exibidas quando alguma pasta ou arquivo não pode ser criado. No caso do Merge Sort, o tempo medido inclui a alocação do vetor auxiliar, por ela fazer parte do algoritmo.

```
bool criarPasta(const string &caminho) {
#if defined(_WIN32)
    int resultado = _mkdir(caminho.c_str());
#else
    int resultado = mkdir(caminho.c_str(), 0777);
#endif
```

```
    return resultado == 0 || errno == EEXIST;
}

bool criarPastas(const string &caminho) {
    for (size_t i = 0; i <= caminho.size(); i++) {
        if (i == caminho.size() || caminho[i] == '/') {
            if (i > 0 && !criarPasta(caminho.substr(0, i)))
                return false;
        }
    }
    return true;
}

bool criarEstruturaPastas(const string &algoritmo) {
    const char *grupos[3] = {"Arquivos de Entrada",
                             "Arquivos de Saida",
                             "Arquivos de Tempo"};
    const char *categorias[3] = {"Crescente", "Decrescente", "Random"};
    for (int g = 0; g < 3; g++) {
        for (int c = 0; c < 3; c++) {
            string caminho = algoritmo + "/" + grupos[g] +
                             "/" + categorias[c];
            if (!criarPastas(caminho)) return false;
        }
    }
    return true;
}

bool operacoes(const string &algoritmo, FuncaoOrdenacao funcao,
               int tipo, int tamanho) {
    if (!criarEstruturaPastas(algoritmo)) return false;
```

```
    clock_t start_t, end_t;
    double tempoGasto;

    int *vetor = gerarSequencia(tipo, tamanho);

    if (!salvarEntrada(algoritmo, tipo, tamanho, vetor)) {
        delete[] vetor;
        return false;
    }

    start_t = clock();
    funcao(vetor, tamanho);
    end_t = clock();

    tempoGasto = (end_t - start_t) / (double)CLOCKS_PER_SEC;

    bool gravou = salvarTempo(algoritmo, tipo, tamanho, tempoGasto) &&
                  salvarSaida(algoritmo, tipo, tamanho, vetor);

    delete[] vetor;
    return gravou;
}
```